

DHBW Stuttgart
Campus Horb

Studienarbeit

Konzeption und prototypische Ansteuerung von GraviTrax Power Connect mit Python

Architektur, Kommunikationsmodell und Entwicklung eines
softwaregestuetzten Demonstrators

Studiengang:	Informatik
Kurs:	TINF23A
Autor:	Max Mustermann
Matrikelnummer:	1234567
Unternehmen:	Musterfirma GmbH
Erstpruefer:	Prof. Dr. Vorname Nachname
Zweitpruefer:	Vorname Nachname
Abgabedatum:	17.03.2026

Inhaltsverzeichnis

1. Einleitung	1
1.1. Motivation	1
1.2. Zielsetzung	2
1.3. Problemstellung	2
1.4. Forschungsfragen	3
1.5. Abgrenzung	3
1.6. Wissenschaftlicher und praktischer Beitrag	4
1.7. Aufbau der Arbeit	4
2. Grundlagen	6
2.1. Fachlicher Kontext	6
2.2. Einordnung als cyber-physisches System	7
2.3. Systemkomponenten und Rollenmodell	7
2.4. Bluetooth Low Energy als Kommunikationsgrundlage	8
2.5. Asynchrone Programmierung mit Python	8
2.6. Funktionale Sicht auf die Bibliothek	9
2.7. Aufbau der Python-Bibliothek	10
2.7.1. Kernmodul <code>gravitrax_bridge</code>	10
2.7.2. Rolle von <code>gravitrax_constants</code>	11
2.7.3. Beispiel- und Applikationsebene	11
2.7.4. Analyse und Bewertung des Aufbaus	12
2.8. Signalmodell und Semantik	12
2.9. Betriebsmodi und Steuerungsstrategien	13
2.10. Softwarequalitaet im Prototyping-Kontext	13
2.11. Zwischenfazit	14
3. Methodik	15
3.1. Vorgehensmodell	15
3.2. Durchfuehrung	15
3.3. Anforderungsanalyse	17
3.3.1. Funktionale Anforderungen	17
3.3.2. Nichtfunktionale Anforderungen	18

3.4.	Architekturentwurf	18
3.5.	Ereignis- und Zustandsmodell	19
3.6.	Regelmodell fuer die Bahnsteuerung	19
3.7.	Implementierungskonzept	19
3.8.	Test- und Evaluationskonzept	20
3.9.	Datenbasis und Dokumentation	20
3.10.	Guetekriterien und methodische Reflexion	20
4.	Ergebnisse und Diskussion	22
4.1.	Ergebnisse	22
4.1.1.	Ergebnisbeitrag zu Forschungsfrage F1	23
4.1.2.	Ergebnisbeitrag zu Forschungsfrage F2	23
4.1.3.	Ergebnisbeitrag zu Forschungsfrage F3	23
4.2.	Ergebnisse aus Szenariotests	24
4.3.	Bewertung entlang der Qualitaetskriterien	25
4.3.1.	Robustheit	25
4.3.2.	Nachvollziehbarkeit	25
4.3.3.	Wartbarkeit	26
4.3.4.	Erweiterbarkeit	26
4.3.5.	Reproduzierbarkeit	26
4.4.	Potenzial fuer die Vermittlung von Programmierung	26
4.4.1.	Didaktische Chancen	27
4.4.2.	Didaktische Risiken und Grenzen	27
4.4.3.	Geeignete Lernziele	28
4.4.4.	Bewertung der didaktischen Eignung	28
4.4.5.	Beispiel fuer eine didaktische Lernprogression	28
4.5.	Konkrete Aufbauten als Anwendungsszenarien	29
4.5.1.	Aufbau 1: Farbstart mit Signalfolge	29
4.5.2.	Aufbau 2: Reaktionsstrecke mit Notification-Callback	29
4.5.3.	Aufbau 3: Zeitmessung zwischen Start und Ziel	30
4.5.4.	Vergleich der Aufbauten	30
4.6.	Diskussion	30
4.6.1.	Kritische Einordnung der Aussagekraft	31
4.6.2.	Risiken und Gegenmassnahmen im Betrieb	32
4.6.3.	Praxisempfehlungen	32
5.	Fazit und Ausblick	33
5.1.	Zusammenfassung	33
5.2.	Beantwortung der Forschungsfragen	33

5.3. Implikationen fuer Praxis und Lehre	34
5.4. Grenzen der Arbeit	34
5.5. Ausblick	35
A. Anhang	36
A.1. Erweitertes Architekturbeispiel (Pseudocode)	37
A.2. Beispiel fuer ein Regelset	39
A.3. Testprotokollvorlage	39
A.4. Beispielhafte Risikoanalyse	40
A.5. Glossar zentraler Begriffe	41
A.6. Erweiterte Codestrecke fuer Logging und Recovery	41
A.7. Konkreter Aufbau 1: Farbstart mit Signalfolge	42
A.8. Konkreter Aufbau 2: Reaktionsstrecke mit Callback	43
A.9. Konkreter Aufbau 3: Zeitmessung zwischen Start und Ziel	45
A.10. Ausfuehrliches Testjournal (erweiterte Stichprobe)	47
A.11. Signalereignisprotokoll (Beispieldatensatz)	52
A.12. Didaktische Zuordnung der Aufbauten	53
A.13. Beispiel fuer eine Unterrichtssequenz	54
Literaturverzeichnis	56

1. Einleitung

1.1. Motivation

Physische Bausysteme gewinnen in der informatischen Bildung und in der prototypischen Produktentwicklung an Bedeutung, wenn sie digitale Steuerung mit unmittelbar sichtbaren Effekten verbinden. Die GraviTrax Murmelbahn von Ravensburger ist ein Beispiel fuer ein modulares System, das durch mechanische Bausteine, Erweiterungen und elektronische POWER-Elemente eine Bruecke zwischen klassischem Konstruktionspielzeug und softwaregestuetzten Interaktionssystemen schlaegt. Mit dem Element Connect wird es moeglich, Signale zwischen physischen Bausteinen und einer Software auszutauschen. Ravensburger stellt hierzu eine Python-Bibliothek bereit, die die Kommunikation mit dem Connect-Stein ueber Bluetooth Low Energy kapselt und damit den Einstieg in eigene Steuerungslogiken erleichtert [8, 10].

Aus technischer Sicht ist dieses Szenario aus mehreren Gruenden relevant. Erstens laesst sich anhand eines ueberschaubaren, aber realen Systems nachvollziehen, wie eingebettete Komponenten, drahtlose Kommunikation und asynchrone Software zusammenspielen. Zweitens bietet die offene Python-Schnittstelle die Moeglichkeit, aus einer spielerischen Anwendung heraus Themen wie Ereignisverarbeitung, Signalprotokolle, Zustandsmodellierung und Hardwareabstraktion zu betrachten. Drittens eignet sich die Plattform fuer Demonstratoren, die im Bildungs-, Messe- oder Prototyping-Kontext anschaulich zeigen, wie digitale Logik physische Prozesse beeinflussen kann.

Aus unternehmerischer Perspektive steigt gleichzeitig der Bedarf an leichtgewichtigen cyber-physischen Demonstratoren. In Innovationsprojekten muessen Softwareideen oft in kurzer Zeit als erlebbarer Prototyp praesentiert werden, bevor eine aufwaendige Produktentwicklung gerechtfertigt ist. Systeme wie GraviTrax POWER Connect sind dafuer attraktiv, weil sie einerseits einen hohen Interaktionsgrad bieten und andererseits mit Standardwerkzeugen wie Python erschlossen werden koennen. Damit verschiebt sich die Frage von „Ist eine technische Umsetzung grundsaeztlich moeglich?“ hin zu „Wie schnell und robust laesst sich die Umsetzung mit vertretbarem Aufwand realisieren?“

Gerade diese Verschiebung ist fuer die Informatik relevant. Statt ein neues Funkprotokoll auf niedriger Ebene zu implementieren, steht die Entwurfsqualitaet einer Anwen-

dung auf Basis einer gegebenen Bibliothek im Vordergrund. Das umfasst Architekturentscheidungen, Ereignismodellierung, Fehlermanagement, Testbarkeit und Wartbarkeit. Die vorliegende Arbeit adressiert somit ein praxisnahes Szenario zwischen Softwaretechnik, eingebetteter Kommunikation und Mensch-Maschine-Interaktion.

1.2. Zielsetzung

Ziel dieser Studienarbeit ist die konzeptionelle und prototypische Untersuchung, wie sich die POWER-Connect-Komponenten von GraviTrax mit Python ansteuern lassen und welche technischen Möglichkeiten sich daraus für eigene Anwendungen ergeben. Im Mittelpunkt stehen dabei nicht der mechanische Aufbau einer konkreten Bahn oder eine spielerische Bewertung des Systems, sondern die softwareseitige Perspektive: Welche Rolle übernimmt der Connect-Stein innerhalb der Kommunikationsarchitektur, welche Funktionen stellt die Bibliothek zur Verfügung und wie kann daraus ein wartbarer Python-Demonstrator abgeleitet werden?

Daraus ergeben sich die folgenden Teilziele:

- Einordnung von GraviTrax POWER und des Connect-Steins in einen technischen Gesamtkontext,
- Analyse der bereitgestellten Python-Bibliothek hinsichtlich Verbindungsaufbau, Signalversand, Ereignisempfang und Geräteinformationen,
- Entwicklung eines prototypischen Nutzungskonzepts für eine softwaregesteuerte Murbelbahn,
- Diskussion der Stärken, Grenzen und Weiterentwicklungspotenziale des Ansatzes.

Die Arbeit verfolgt damit einen anwendungsorientierten Fokus. Es wird kein neues Kommunikationsprotokoll entwickelt. Stattdessen wird untersucht, wie sich die vorhandene Bibliothek zielgerichtet einsetzen lässt, um bestehende GraviTrax-POWER-Funktionen aus einer Python-Anwendung heraus nutzbar zu machen.

1.3. Problemstellung

Obwohl die Bibliothek für GraviTrax POWER Connect offen zugänglich ist, ist der Weg von einem lauffähigen Beispielskript zu einer belastbaren Anwendung nicht trivial. In der Praxis treten typischerweise mehrere Herausforderungen auf:

- Die Verbindung zu BLE-Geraeten ist zustandsbehaftet und kann durch Interferenzen, Reichweitenprobleme oder Betriebssystembesonderheiten unterbrochen werden.
- Die Semantik empfangener Signale muss fachlich interpretiert werden; eine rein technische Verarbeitung reicht fuer verstaendliche Anwendungslogik nicht aus.
- Asynchrone Ablaefue erhoehen die Komplexitaet beim Zusammenspiel von Benutzeroberflaeche, Zeitlogik und Hardwarekommunikation.
- Fuer Demonstratoren im Bildungs- oder Praesentationskontext ist nicht nur Funktionalitaet, sondern auch reproduzierbares Verhalten entscheidend.

Aus dieser Problemstellung ergibt sich der Bedarf nach einem strukturierten Vorgehen, das die Bibliothek nicht nur funktional verwendet, sondern in eine saubere Softwarearchitektur einbettet. Die Arbeit liefert hierzu ein methodisches und technisches Referenzmodell.

1.4. Forschungsfragen

Zur Operationalisierung der Zielsetzung werden drei zentrale Forschungsfragen (F1–F3) formuliert:

- **F1:** Welche Funktionen und Grenzen weist die GraviTrax-Connect-Python-Bibliothek fuer prototypische Steuerungsaufgaben auf?
- **F2:** Wie kann eine wartbare Anwendungsarchitektur gestaltet werden, die BLE-Kommunikation, Signalverarbeitung und Fachlogik sauber trennt?
- **F3:** Welche Qualitaetskriterien sind fuer den praktischen Einsatz eines software-gesteuerten Murrelbahn-Demonstrators besonders relevant?

Die Forschungsfragen verbinden eine technologieorientierte Analyse mit software-technischen Entwurfszielen. Im Ergebnis soll nicht nur „ob“ die Ansteuerung moeglich ist, sondern auch „wie“ sie nachhaltig umgesetzt werden kann.

1.5. Abgrenzung

Die Arbeit grenzt den Untersuchungsgegenstand bewusst ein, um eine belastbare Tiefe zu erreichen:

- Untersucht wird die Python-basierte Ansteuerung ueber den Connect-Stein, nicht die Entwicklung eigener Firmware fuer POWER-Steine.
- Der Schwerpunkt liegt auf Konzeption, Architektur und prototypischer Umsetzung; eine industrielle Produktreife wird nicht angestrebt.
- Quantitative Funkmessungen (z. B. detaillierte Spektrumanalysen) sind nicht Kern der Arbeit, werden jedoch qualitativ eingeordnet.
- Die didaktische Bewertung im Unterrichtskontext wird diskutiert, aber nicht durch eine empirische Lernstudie validiert.

Diese Abgrenzung ist notwendig, um den Umfang der Studienarbeit beherrschbar zu halten und gleichzeitig einen hohen fachlichen Mehrwert zu erzielen.

1.6. Wissenschaftlicher und praktischer Beitrag

Die Arbeit leistet einen doppelten Beitrag. Wissenschaftlich wird ein strukturierter Bezugsrahmen fuer ein konkretes cyber-physisches Anwendungsszenario bereitgestellt. Praktisch entsteht ein nachvollziehbares Vorgehensmodell fuer die Entwicklung eines Demonstrators auf Basis der offiziellen Bibliothek. Der Mehrwert zeigt sich insbesondere in folgenden Punkten:

- systematische Zusammenfuehrung von Hardware-, Protokoll- und Softwareperspektive,
- uebertragbares Architekturmodell fuer ereignisgetriebene BLE-Anwendungen,
- konkrete Kriterien zur Bewertung von Robustheit und Wartbarkeit,
- praxisnahe Ableitung von Empfehlungen fuer weitere Projekte.

Damit kann die Arbeit als Ausgangspunkt fuer Folgearbeiten dienen, in denen entweder eine technische Vertiefung (z. B. Timing-Analyse) oder eine didaktische Vertiefung (z. B. Lernsettings) vorgenommen wird.

1.7. Aufbau der Arbeit

Kapitel 2 beschreibt die fachlichen und technischen Grundlagen. Dazu gehoeren der Aufbau des GraviTrax-POWER-Systems, die Rolle des Connect-Steins, Grundzuege

von Bluetooth Low Energy sowie die Bedeutung asynchroner Programmierung mit Python. Kapitel 3 stellt die Methodik und das Vorgehen fuer die prototypische Ansteuerung dar. Dabei werden Anforderungen, Systementwurf und exemplarische Implementierungsschritte beschrieben. Kapitel 4 fasst die Ergebnisse der Analyse zusammen und diskutiert den praktischen Nutzen sowie die Grenzen der Bibliothek. Kapitel 5 schliesst die Arbeit mit einem Fazit und einem Ausblick auf moegliche Erweiterungen ab.

2. Grundlagen

2.1. Fachlicher Kontext

GraviTrax ist ein modulares Kugelbahn-System, bei dem aus Basiselementen, Höhensteinen, Schienen und Funktionselementen individuelle Bahnverläufe aufgebaut werden. Mit der Produktlinie POWER wird das System um elektronische Steine erweitert, die auf Funksignale reagieren oder selbst Signale auslösen. Dadurch verlagert sich der Fokus von einer rein mechanischen Bahn hin zu einem hybriden System, in dem physische Bewegung, diskrete Ereignisse und elektronische Schaltlogik kombiniert werden. Der Connect-Stein nimmt in diesem Ökosystem eine Sonderrolle ein, da er als Bluetooth-fähige Brücke zwischen externer Software und den POWER-RF-Steinen dient [8].

Aus Sicht der Informatik ist diese Architektur deshalb interessant, weil sie ein klar abgegrenztes cyber-physisches System repräsentiert. Die mechanische Domäne besteht aus der Bahn und den Murmeln, die digitale Domäne aus Signalfunktionen, Farbkanaelen, Sensordaten und Steuerbefehlen. Zwischen beiden Ebenen steht ein Ereignismodell: Eine Murmel löst an einem Stein einen Zustand aus, der als digitales Signal weitergeleitet und wiederum in eine physische Aktion übersetzt werden kann. Solche Ketten sind typisch für interaktive Systeme, Robotik-Demonstratoren oder Lernumgebungen, in denen abstrakte Softwarekonzepte in der realen Welt beobachtbar werden.

Die von Ravensburger veröffentlichte Python-Bibliothek ist explizit darauf ausgelegt, mit einem GraviTrax Bluetooth Connect stone zu kommunizieren, welcher seinerseits die POWER-RF-Steine ansteuert. Laut Projektbeschreibung können mit der Bibliothek Signale empfangen, Signale gesendet, Statusinformationen der Bridge ausgelesen und Beispielanwendungen wie eine Kommandozeilen- oder GUI-Steuerung umgesetzt werden [8]. Damit existiert eine für Studien- und Prototyping-Zwecke geeignete Softwarebasis.

2.2. Einordnung als cyber-physisches System

Cyber-physische Systeme (CPS) sind dadurch gekennzeichnet, dass informationstechnische und physische Prozesse eng gekoppelt sind. Sensorik erfasst Zustände der realen Welt, Software verarbeitet diese Informationen, und Aktorik erzeugt wiederum physische Effekte. Genau dieses Muster lässt sich bei GraviTrax POWER Connect nachvollziehen: Das System erfasst Ereignisse in der Bahn, überträgt sie als digitale Signale und kann durch softwareseitige Befehle neue physische Reaktionen auslösen [5].

Die Relevanz für diese Arbeit liegt in der klaren Nachvollziehbarkeit der Kette aus Beobachtung, Entscheidung und Aktion. In vielen industriellen CPS-Anwendungen ist diese Kette durch komplexe Anlagenstrukturen schwer zu isolieren. Bei der Marmelbahn sind die Zusammenhänge direkt sichtbar. Dadurch eignet sich das System besonders für die Untersuchung grundlegender Fragen zu Reaktionszeiten, Ereignisverdichtung, Fehlerbehandlung und Zustandskonsistenz.

2.3. Systemkomponenten und Rollenmodell

Für die softwaretechnische Analyse ist ein Rollenmodell hilfreich. Tabelle 2.1 differenziert die relevanten Komponenten.

komponente	Rolle im Gesamtsystem
Marmelbahnaufbau	Physische Ausführungsebene; erzeugt reale Ereignisse durch Bewegung und Kontakt.
POWER-RF-Steine	Empfangen und senden Funksignale innerhalb der Bahnlogik; setzen Signale in lokale Aktionen um.
Connect-Stein	Brücke zwischen externer Software und POWER-Funknetz; bietet den BLE-Zugangspunkt für die Anwendung.
Python-Bibliothek	Abstraktion der Kommunikationsdetails; stellt Funktionen für Verbindung, Signale, Notifications und Diagnose bereit.
Anwendungslogik	Fachliche Regeln, z. B. welche Reaktion auf welche Ereigniskombination folgt.

Tabelle 2.1.: Rollenmodell der untersuchten Architektur

Das Rollenmodell macht deutlich, dass die Bibliothek nicht das Gesamtsystem ersetzt, sondern eine mittlere Integrationsschicht bildet. Gerade diese Position ist kritisch: Ist die Abstraktion zu niedrig, steigt die Komplexität der Anwendung. Ist sie zu hoch, gehen Steuerungsmöglichkeiten verloren. Die dokumentierte API trifft für prototypische Zwecke einen praxisnahen Mittelweg [10, 1].

2.4. Bluetooth Low Energy als Kommunikationsgrundlage

Die technische Grundlage der Python-Bibliothek bildet Bluetooth Low Energy (BLE). BLE ist fuer energiearme, paketerorientierte Kommunikation zwischen Geraeten mit vergleichsweise geringen Datenraten ausgelegt. Im vorliegenden Szenario nutzt der Connect-Stein BLE fuer die Verbindung zum Rechner oder zu einem Einplatinencomputer. Die eigentliche Bibliothek verwendet wiederum das Python-Paket bleak als plattformuebergreifende Abstraktion fuer BLE-Zugriffe unter Windows und Linux [2, 8]. Dadurch muss die Anwendungslogik nicht direkt mit Betriebssystem-APIs arbeiten. Funktional relevant sind insbesondere drei BLE-Mechanismen:

- **Scanning:** Ermittlung erreichbarer Geraete und Auswahl der Ziel-Bridge.
- **GATT-basierte Kommunikation:** Lesen, Schreiben und Benachrichtigung ueber Characteristic-Werte.
- **Notification-Streams:** Eventgetriebener Empfang von Signalen ohne polling-basierte Dauerabfragen.

Fuer Demonstratoren ist BLE vor allem wegen des geringen Integrationsaufwands sinnvoll. Eine USB-spezifische oder proprietäre Funkanbindung waere fuer studentische Projekte oft aufwaendiger. Gleichzeitig fuehrt BLE zu typischen Risiken wie Verbindungsabbruechen, variablem Pairing-Verhalten und plattformabhaengigen Besonderheiten. Daraus ergibt sich die Notwendigkeit robuster Zustandsuebergaenge in der Anwendung.

2.5. Asynchrone Programmierung mit Python

Ein zweiter technischer Kern ist das asynchrone Programmiermodell. Die Dokumentation der Bibliothek weist darauf hin, dass die meisten Methoden der Klasse Bridge als Coroutines ausgefuehrt werden und daher in einer asynchronen Funktion mit `asyncio` sowie dem Schluesselwort `await` aufgerufen werden muessen [10, 7]. Dieses Modell ist fuer die Ansteuerung eines BLE-Geraets sinnvoll, weil Verbindungsaufbau, Scans, Benachrichtigungen und Signalversand nebenlaeufige Wartezustaende enthalten.

Im Unterschied zu klassischen Thread-Modellen konzentriert `asyncio` konkurrierende Aufgaben auf einen Event Loop. Das reduziert den Synchronisationsaufwand, setzt aber saubere Strukturierung voraus. Fuer die vorliegende Anwendung bedeutet das konkret:

- Benachrichtigungs-Callbacks muessen kurz und deterministisch bleiben.

- Zeitintensive Verarbeitung sollte in separate Tasks ausgelagert werden.
- Gemeinsame Zustände sind über klar definierte Schnittstellen zu aktualisieren.
- Fehler in Tasks dürfen nicht stillschweigend verloren gehen.

Bei korrekter Umsetzung ermöglicht asyncio eine hohe Reaktionsfähigkeit der Anwendung selbst dann, wenn parallel GUI-Ereignisse, Timer und BLE-Nachrichten verarbeitet werden. Damit passt das Modell gut zu interaktiven Demonstratoren.

2.6. Funktionale Sicht auf die Bibliothek

Die von Ravensburger dokumentierten Kernfunktionen der Bibliothek lassen sich in vier Funktionsbereiche einteilen:

oprüle Funktionsbereich	Beschreibung
Verbindungsmanagement	Aufbau und Trennung einer BLE-Verbindung zum Connect-Stein, optional mit Wiederverbindungslogik und Callback bei Verbindungsabbruechen.
Signalsteuerung	Senden einzelner oder periodischer Signale an POWER-Steine, optional mit Resends und parametrisierten Farb- und Statuswerten.
Ereignisempfang	Aktivierung von Notifications, Auswertung empfangener Signale und Weitergabe der Daten an eigene Callback-Funktionen.
Geraeteinformationen	Auslesen von Batteriestand, Firmware- und Hardwareversion sowie Hilfsfunktionen wie Scan und Logging.

Tabelle 2.2.: Zentrale Funktionsbereiche der GraviTrax-Connect-Bibliothek nach der offiziellen Dokumentation [10]

Fuer den praktischen Einsatz ist insbesondere die Klasse Bridge relevant. Ein minimales Beispiel besteht aus der Instanziierung eines Bridge-Objekts und dem anschliessenden Aufruf von connect() innerhalb eines asyncio-Hauptprogramms [10]. Darauf aufbauend koennen Notifications aktiviert, Signale an definierte Farbkanaele gesendet oder Batteriestand und Firmware abgefragt werden.

Aus Architektursicht ist hervorzuheben, dass die Bibliothek semantische Parameter wie „Stone“, „Color“ und „Status“ bereitstellt. Dadurch bleibt die Fachlogik auf einer hoeheren Abstraktionsebene und muss nicht direkt auf Bytepositionen operieren. Fuer die Wartbarkeit ist diese Trennung zentral.

2.7. Aufbau der Python-Bibliothek

Fuer die vorliegende Arbeit ist nicht nur die Benutzung der Bibliothek relevant, sondern auch ihr interner Aufbau. Die Repository-Struktur zeigt eine fuer Lehr- und Prototyping-Projekte typische Dreiteilung: einen kompakten Bibliothekskern, eine Sammlung kleiner Beispielskripte und mehrere groessere Anwendungsprogramme [8]. Der eigentliche Python-Paketordner „gravitraxconnect“ exportiert ueber „__init__.py“ vor allem die Module „gravitrax_bridge“ und „gravitrax_constants“. Dadurch bleibt der Einstieg fuer Anwender einfach, weil die zentralen Schnittstellen bereits auf Paketebene sichtbar sind.

Die Struktur kann wie folgt zusammengefasst werden:

Bestandteil	Funktion im Projekt
<code>gravitrax_bridge.py</code>	Zentrales Kernmodul mit der Klasse <code>Bridge</code> , BLE-Kommunikation, Notification-Handling, Signalversand und Hilfsfunktionen.
<code>gravitrax_constants.py</code>	Sammlung technischer Konstanten wie UUIDs, Default-Werte, Farb-, Status- und Steinszuordnungen.
<code>examples/</code>	Kleine, fokussierte Beispielskripte fuer einzelne Anwendungsfaelle wie Connect, Notifications, Bridge-Mode oder Signalwiederholung.
<code>Applications/</code>	Groessere Demonstratoranwendungen wie CLI, GUI, Raspberry-Pi-GPIO und Timer.
<code>tests/</code>	Funktionale Tests fuer Verbindungsaufbau, Signalversand, Recovery und Laufzeitverhalten.

Tabelle 2.3.: Struktureller Aufbau des GraviTrax-Connect-Repositories nach der offiziellen Projektstruktur [8]

Diese Struktur ist aus softwaretechnischer Sicht sinnvoll. Sie trennt den wiederverwendbaren Bibliothekskern von Anwendungs- und Demonstrationscode. Besonders positiv ist, dass die offiziellen Beispiele genau die Lernschritte abbilden, die ein Nutzer typischerweise durchlauft: Zunaechst wird die Bridge gefunden und verbunden, danach werden Notifications aktiviert, anschliessend werden eigene Reaktionen oder Diagnosefunktionen implementiert. Dadurch entsteht nicht nur eine technische, sondern auch eine didaktische Progression.

2.7.1. Kernmodul `gravitrax_bridge`

Das Modul `gravitrax_bridge.py` bildet das Herzstueck der Bibliothek. Laut Quelltext kapselt die Klasse `Bridge` sowohl den Zustand der aktuellen Verbindung als auch die zentralen Aktionen wie `connect()`, `disconnect()`, `notification_enable()`,

`send_signal()` oder `request_bridge_info()` [8]. Darueber hinaus enthaelt das Modul Hilfsfunktionen wie `scan_bridges()`, `calc_checksum()`, `add_checksum()` sowie Logging-Unterstuetzung.

Die interne Gestaltung weist mehrere interessante Eigenschaften auf:

- Verbindungsbezogene Zustandsdaten wie Name, Adresse, Firmware, Hardware und Wiederverbindungsparameter werden im `Bridge`-Objekt gebuendelt.
- Fuer kritische Abschnitte verwendet die Bibliothek `asyncio.Lock`, etwa zur Kontrolle eingehender Notifications und der naechsten Send-ID.
- Der Notification-Handler prueft eingehende Daten auf gueltige Signalstruktur, filtert Dubletten und verwirft Nachrichten mit falscher Checksumme [8].
- Disconnects koennen ueber Callback-Funktionen sowie optionales Reconnect-Verhalten behandelt werden.

Diese Punkte sprechen fuer einen robusten, praxisorientierten Entwurf. Die Entwickler haben nicht nur die Happy-Path-Nutzung implementiert, sondern auch typische Laufzeitprobleme wie doppelte Notifications, Integritaetspruefung und Verbindungsabbrueche beruecksichtigt.

2.7.2. Rolle von `gravitrax_constants`

Das Modul `gravitrax_constants.py` wirkt auf den ersten Blick unspektakulaer, ist fuer die Qualitaet der Anwendungsentwicklung jedoch zentral. Dort sind BLE-spezifische UUIDs, Standardwerte fuer Nachrichtfelder, Namen der Bridge sowie Nachschlagetabellen fuer Farben, Statuscodes und Steinarten definiert [8]. Die offizielle Dokumentation empfiehlt ausdruuecklich, diese Konstanten in eigenen Programmen zu verwenden, um Lesbarkeit und Zukunftssicherheit zu verbessern [10].

Gerade fuer Studien- und Lehrprojekte ist das wichtig. Anstelle schwer interpretierbarer Zahlenwerte koennen Programme mit semantischen Bezeichnern wie `COLOR_RED`, `STATUS_ALL` oder `STONE_BRIDGE` formuliert werden. Damit sinkt die kognitive Last bei Entwicklung und Wartung deutlich.

2.7.3. Beispiel- und Applikationsebene

Neben dem Bibliothekskern ist die zweite Ebene des Repositories bemerkenswert: die Vielzahl offizieller Beispiele. Dazu gehoeren unter anderem Skripte fuer Notifications, Signalsequenzen, Signalwiederholung, Bridge-Mode, Diagnoseabfragen oder das Verbinden ueber eine MAC-Adresse [8]. Die Anwendungen gehen darueber hinaus und

zeigen, wie sich auf Basis derselben Bibliothek eine CLI, eine GUI, eine Raspberry-Pi-GPIO-Anbindung oder eine Zeitmessanwendung realisieren lassen.

Diese Struktur erlaubt eine wichtige Beobachtung: Die Bibliothek ist klein genug, um ueberschaubar zu bleiben, aber reichhaltig genug, um auf ihrer Basis unterschiedliche Interaktionsmodelle aufzubauen. Das ist ein starkes Indiz fuer gute Wiederverwendbarkeit.

2.7.4. Analyse und Bewertung des Aufbaus

Insgesamt laesst sich der Aufbau der Bibliothek positiv bewerten. Zu den Staerken gehoeren die klare Exportstruktur, die semantisch hilfreichen Konstanten, die praxisnahen Beispielprogramme und die bereits vorhandenen Funktionstests. Die Bibliothek ist damit fuer Einsteiger zugaenglich und zugleich fuer weitergehende Demonstratoren brauchbar.

Es bestehen jedoch auch Schwachstellen. Besonders auffaellig ist die starke Zentralisierung von Verantwortlichkeiten im Modul `gravitrax_bridge.py`. Verbindung, Protokollverarbeitung, Fehlerbehandlung, Logging-nahe Hilfen und Teile der Laufzeitsteuerung befinden sich weitgehend in einer Datei. Fuer kleine Bibliotheken ist das vertretbar, mittelfristig schwaecht es jedoch Modularitaet und Testbarkeit. Aus Sicht der Softwarearchitektur waere eine feinere Trennung in Verbindungsadapter, Protokollparser und Sessionsteuerung langfristig wartungsfreundlicher.

Dennoch ueberwiegt fuer den hier betrachteten Einsatzzweck der Nutzen. Die Bibliothek ist nicht uebertechnisiert, sondern pragmatisch entworfen. Gerade fuer Studienarbeiten und Lernprojekte ist diese pragmatische Balance oft sinnvoller als eine sehr stark fragmentierte Architektur.

2.8. Signalmodell und Semantik

Ein softwaregesteuertes Bahnverhalten entsteht aus der Interpretation einzelner Signale. Die dokumentierten Signalparameter beinhalten mindestens Header, Stone, Status, Id, Checksum und Color [10]. Fuer die Anwendung ergibt sich daraus ein semantisches Signalmodell, das in Tabelle 2.4 zusammengefasst ist.

Aus diesen Attributen lassen sich Ereignisregeln ableiten, beispielsweise: „Wenn von Stein X ein rotes Signal mit Status Y eintrifft, sende fuer Stein Z ein gruens Signal mit Status A“. In der Softwaretechnik entspricht dies einem Event-Condition-Action-Muster (ECA), das fuer interaktive Systeme weit verbreitet ist.

oprule Signalattribut	Bedeutung fuer die Fachlogik
Stone	Identifikation des beteiligten Steintyps; dient der Quell- und Zielzuordnung.
Color	Logischer Kanal zur Trennung von Signalpfaden; kann als Zustandsmarker genutzt werden.
Status	Steuerwert fuer den gewuenschten Aktionszustand oder Ereignistyp.
Id	Differenzierung einzelner Signale, hilfreich fuer Korrelation und Logging.
Checksum	Integritaetspruefung auf Protokollebene; relevant fuer Plausibilitaetskontrollen.

Tabelle 2.4.: Semantische Interpretation der Signalattribute

2.9. Betriebsmodi und Steuerungsstrategien

Ein relevanter Aspekt ist der sogenannte Bridge-Only-Mode. In diesem Betriebszustand reagieren empfangende Steine nur noch auf Signale, die explizit vom Bridge-Stein kommen [8, 10]. Damit kann die Software bei Bedarf die Kontrolle ueber den Signalfluss strenger zentralisieren.

Fachlich lassen sich daraus zwei Strategien ableiten:

- **Kooperativer Modus:** Bahninterne und softwareseitige Signale wirken gemeinsam; geeignet fuer explorative Aufbauten.
- **Zentraler Modus:** Die Software orchestriert den Ablauf bewusst; geeignet fuer reproduzierbare Demonstrationen und Messungen.

Der zentrale Modus reduziert unerwartete Kettenreaktionen, erhoehrt jedoch die Verantwortung der Anwendung. Fehler in der Logik koennen den gesamten Ablauf blockieren. Entsprechend wichtig sind Timeouts, Fallback-Signale und ein geordnetes Recovery.

2.10. Softwarequalitaet im Prototyping-Kontext

Auch fuer prototypische Systeme gelten Qualitaetskriterien. Im Rahmen dieser Arbeit stehen folgende Eigenschaften im Vordergrund:

- **Robustheit:** Die Anwendung bleibt bei temporaeren Verbindungsstoerungen kontrollierbar.

- **Nachvollziehbarkeit:** Signalwege und Entscheidungen sind ueber Logs oder Visualisierung rekonstruierbar.
- **Wartbarkeit:** Fachlogik, Infrastruktur und Hardwarezugriff sind klar getrennt.
- **Erweiterbarkeit:** Neue Regeln, weitere Steine oder zusaetzliche Oberflaechen lassen sich ohne Komplettumbau integrieren.
- **Reproduzierbarkeit:** Gleiches Setup fuehrt unter gleichen Bedingungen zu vergleichbaren Ergebnissen.

Diese Kriterien dienen in Kapitel 4 als Bewertungsraster fuer die Diskussion der Ergebnisse.

2.11. Zwischenfazit

Die Grundlagenanalyse zeigt, dass GraviTrax POWER Connect ein geeignetes Untersuchungsfeld fuer ereignisgetriebene, asynchron arbeitende CPS-Anwendungen darstellt. Die Kombination aus BLE-basierter Konnektivitaet, semantisch orientierter Bibliothek und sichtbaren physischen Effekten bietet eine solide Basis fuer den in Kapitel 3 beschriebenen methodischen Entwurf.

3. Methodik

3.1. Vorgehensmodell

Die Bearbeitung des Themas erfolgt als konzeptionell-prototypische Studienarbeit. Im Unterschied zu einer rein theoretischen Literaturarbeit steht die Frage im Zentrum, wie die vorhandene Bibliothek fuer eine konkrete Steuerungsaufgabe strukturiert eingesetzt werden kann. Gleichzeitig wird auf eine vollstaendige Produktentwicklung verzichtet. Das Vorgehensmodell kombiniert daher Dokumentenanalyse, Systementwurf und exemplarische Implementierungsschritte.

Zu Beginn werden die offiziellen Projektquellen ausgewertet. Dazu gehoeren das Repository der Python-Bibliothek, die Installationshinweise und die technische Nutzungsdokumentation [8, 10]. Auf dieser Grundlage werden die verfuegbaren Schnittstellen identifiziert und in funktionale Teilaspekte zerlegt. Anschliessend wird ein Demonstrator konzipiert, der typische Anforderungen an eine softwaregesteuerte Murrenbahn abbildet: Verbindung zum Connect-Stein herstellen, Signale empfangen, auf Ereignisse reagieren und definierte Steuerbefehle aussenden.

Methodisch bietet sich eine schichtenorientierte Betrachtung an. Die unterste Ebene ist die physische Hardware mit Connect- und POWER-Steinen. Darueber liegt die Bibliothek als Kommunikationsabstraktion. Auf oberster Ebene befindet sich die eigentliche Anwendungslogik, die aus fachlichen Regeln wie Start, Reaktion, Wiederholung oder Auswertung besteht. Diese Trennung ist entscheidend, weil sie Wartbarkeit und Erweiterbarkeit verbessert. Die Anwendung soll nicht von BLE-Details oder Byteformaten abhaengen, sondern auf lesbaren Zustands- und Ereignisbegriffen aufsetzen.

3.2. Durchfuehrung

Die prototypische Durchfuehrung beginnt mit den technischen Voraussetzungen. Laut Dokumentation werden mindestens Python 3.10, das Paket bleak sowie unter Windows ein aktuelles Betriebssystem ab Windows 10 Version 1709 benoetigt [8, 9]. Die Bibliothek selbst setzt auf asyncio und kann entweder aus dem Quellverzeichnis installiert oder in ein bestehendes Python-Projekt eingebunden werden. Fuer den Entwurf der Anwendung ist daraus abzuleiten, dass alle zentralen Hardwarezugriffe asynchron

modelliert werden muessen.

Ein plausibler Demonstrator folgt dem in Tabelle 3.1 beschriebenen Ablauf. Ziel ist nicht die Maximierung von Spielkomplexitaet, sondern die transparente Abbildung des Kommunikationsmodells.

Schritt	Inhalt
1	Initialisierung der Anwendung und Erzeugung eines Bridge-Objekts.
2	Aufbau der BLE-Verbindung zum Connect-Stein mittels <code>connect()</code> .
3	Aktivierung von Notifications mit einer eigenen Callback-Funktion.
4	Auswertung eingehender Signale nach Stein-Typ, Farbe und Status.
5	Reaktion durch den Versand eines passenden Steuersignals mit <code>send_signal()</code> oder <code>send_periodic()</code> .
6	Optionales Auslesen von Batteriestand oder Firmware fuer Diagnosezwecke.
7	Sauberes Beenden der Session ueber <code>disconnect()</code> .

Tabelle 3.1.: Ablauf eines prototypischen Python-Demonstrators

Die eigentliche fachliche Logik kann ereignisgetrieben umgesetzt werden. Sobald die Bibliothek eine Notification liefert, werden die uebergebenen Signalparameter geprueft. Die Dokumentation nennt unter anderem Header, Stone, Status, Id, Checksum und Color als auswertbare Werte [10]. Auf dieser Grundlage laesst sich eine Regel formulieren wie: Wenn ein bestimmter Stein ein rotes Signal mit definiertem Status sendet, dann loese als Reaktion ein gruenes Signal fuer einen anderen Empfaenger aus. Eine solche Logik bildet einen typischen Event-Handler und macht das System didaktisch gut nachvollziehbar.

Aus Softwarearchitektursicht empfiehlt sich fuer den Demonstrator eine Aufteilung in mindestens drei Module:

- ein Verbindungsmodul fuer Aufbau, Wiederverbindung und Logging,
- ein Protokoll- oder Signalmodul fuer die fachliche Interpretation von Farben, Statuswerten und Stein-Typen,
- ein Anwendungsmodul mit den eigentlichen Regeln der Bahnsteuerung.

Durch diese Trennung bleibt die Anwendungslogik auch dann lesbar, wenn spaeter eine grafische Oberflaeche, Zeiterfassung oder ein Mehrbahn-Szenario ergaenzt werden soll. Die Dokumentation des Repositories liefert bereits Hinweise auf weitergehende Beispielanwendungen wie CLI, GUI, GPIO-Anbindung fuer den Raspberry Pi und einen Timer [8]. Diese Beispiele bestaetigen, dass die Bibliothek fuer unterschiedliche Nutzungskontexte ausgelegt ist.

Fuer die Validierung des Prototyps stehen in erster Linie funktionale Kriterien im Vordergrund. Dazu gehoert, ob eine stabile Verbindung aufgebaut werden kann, ob Notifications reproduzierbar eintreffen, ob Signale mit der erwarteten Semantik versandt werden und ob Diagnoseinformationen abrufbar sind. Eine tiefergehende quantitative Messung, etwa zu Latenzen oder Funkreichweiten, waere moeglich, ist jedoch nicht Kern dieser Arbeit. Stattdessen liegt der Schwerpunkt auf der Frage, ob die Bibliothek fuer prototypische Steuerungsaufgaben hinreichend abstrahiert und praktikabel ist.

3.3. Anforderungsanalyse

Zur methodischen Absicherung wird das Vorgehen durch eine strukturierte Anforderungsanalyse ergaenzt. Diese differenziert funktionale und nichtfunktionale Anforderungen. Funktionale Anforderungen beschreiben, *was* das System leisten muss, nichtfunktionale Anforderungen beschreiben, *wie gut* dies unter gegebenen Randbedingungen erreicht werden soll.

3.3.1. Funktionale Anforderungen

Aus den Forschungsfragen werden folgende Kernanforderungen (FA1–FA8) abgeleitet:

- **FA1:** Die Anwendung muss eine Verbindung zum Connect-Stein aufbauen und kontrolliert trennen koennen.
- **FA2:** Eingehende Signale muessen in Echtzeit als Ereignisse verarbeitet werden.
- **FA3:** Die Anwendung muss auf Ereignisse mit regelbasierten Ausgangssignalen reagieren koennen.
- **FA4:** Es muss moeglich sein, periodische Signale zu senden und wieder zu stoppen.
- **FA5:** Verbindungsabbrueche muessen erkannt und in definierte Wiederanlaufstrategien ueberfuehrt werden.
- **FA6:** Diagnosedaten (z. B. Batteriestatus, Firmware) muessen abrufbar sein.
- **FA7:** Das System muss protokollieren, welche Signale empfangen und gesendet wurden.
- **FA8:** Die Regeldefinition muss ohne Aenderung der Infrastrukturkomponenten erweiterbar sein.

3.3.2. Nichtfunktionale Anforderungen

Die nichtfunktionalen Anforderungen bilden das Qualitätsgerüst des Demonstrators:

- **NFA1 Robustheit:** Der Demonstrator darf bei kurzzeitigen Störungen nicht in einen undefinierten Zustand fallen.
- **NFA2 Verständlichkeit:** Architektur und Quellcode müssen für Dritte nachvollziehbar dokumentiert sein.
- **NFA3 Reaktionsfähigkeit:** Ereignisse sollen mit geringer, subjektiv kaum wahrnehmbarer Verzögerung verarbeitet werden.
- **NFA4 Portabilität:** Das Konzept soll auf den in der Bibliothek vorgesehenen Plattformen reproduzierbar sein.
- **NFA5 Wartbarkeit:** Änderungen an Regeln sollen ohne tiefen Eingriff in BLE-spezifischen Code möglich sein.

3.4. Architekturentwurf

Auf Basis der Anforderungen wird eine vierstufige Referenzarchitektur vorgeschlagen. Sie trennt Zustandsmanagement, Kommunikation und Fachregeln, um Kopplung zu reduzieren.

Schicht	Verantwortung
Presentation	Optionale Benutzerinteraktion (CLI/GUI), Start-Stopp-Steuerung, Statusanzeige.
Application	Ablaufkoordination, Regelengine, Fehlerbehandlung, Session-Lebenszyklus.
Integration	Adapter zur Bridge-API, Mapping von Rohdaten in domänenspezifische Ereignisse.
Infrastructure	Betriebssystemnahe BLE-Laufzeit über bleak, Event Loop, Logging-Backend.

Tabelle 3.2.: Referenzarchitektur für den Demonstrator

Die Integrationsschicht ist besonders wichtig, weil sie die direkte Abhängigkeit zur Bibliothek kapselt. Sollte sich die API in einer späteren Version ändern, ist idealerweise nur diese Schicht betroffen. Die Fachlogik bleibt dagegen weitgehend stabil.

3.5. Ereignis- und Zustandsmodell

Zur strukturierten Verarbeitung wird ein explizites Zustandsmodell fuer die Session eingefuehrt. Ein moeglicher Zustandsautomat umfasst die Zustaende *Idle*, *Connecting*, *Connected*, *Recovering* und *Stopping*. Ereignisse wie *connect erfolgreich*, *disconnect erkannt* oder *timeout* fuehren zu definierten Uebergaengen.

Ein solches Modell verhindert, dass Signale in inkonsistenten Phasen gesendet werden, beispielsweise waehrend bereits ein Reconnect laeuft. In asynchronen Anwendungen ist dies ein haeufiger Fehler. Durch klare Zustandsregeln wird die Komplexitaet beherrschbar gehalten.

3.6. Regelmodell fuer die Bahnsteuerung

Die eigentliche Steuerung wird als Menge deklarativer Regeln modelliert. Jede Regel besteht aus Trigger, Bedingung und Aktion:

- **Trigger:** Eingangssignal mit Attributen wie Stone, Color, Status.
- **Bedingung:** Optionaler Kontext, z. B. nur in Betriebsmodus „Demo-A“.
- **Aktion:** Versand eines oder mehrerer Ausgangssignale, ggf. mit Verzoegerung.

Die Trennung zwischen Regeldefinition und Ausfuehrungslogik erhoehrt die Anpassbarkeit. Neue Demonstrationsszenarien koennen so durch Konfigurationsaenderung statt durch tiefgreifenden Codeumbau entstehen.

3.7. Implementierungskonzept

Die prototypische Implementierung orientiert sich an den in der Bibliothek dokumentierten API-Funktionen [10]. Dabei werden folgende Entwurfsmuster genutzt [3, 1]:

- **Adaptermuster** fuer den Zugriff auf Bridge-Funktionen.
- **Observermuster** fuer Notification-basierte Ereignisverarbeitung.
- **Command-Pattern** fuer kapselbare Sendeoperationen.
- **Retry-Strategie** mit begrenzter Wiederholungszahl fuer Verbindungsprobleme.

Ein zentrales Designziel ist die Wiederverwendbarkeit. Die gleichen Kernmodule sollen in einem CLI-Tool, einer GUI oder einem Headless-Demonstrator auf Einplatinenhardware einsetzbar sein.

3.8. Test- und Evaluationskonzept

Da keine industrielle Produktvalidierung angestrebt wird, wird ein pragmatisches, aber systematisches Testkonzept genutzt. Es umfasst vier Ebenen:

- **T1 Modultests:** Validierung einzelner Logikkomponenten mit simulierten Signalen.
- **T2 Integrationstests:** Zusammenspiel von Anwendung und Bridge-API unter Realbedingungen.
- **T3 Szenariotests:** Wiederholte Durchläufe vordefinierter Bahnabläufe.
- **T4 Fehlerszenarien:** Gezielte Störungen wie Deaktivieren der Bridge oder erzwungene Disconnects.

Für jedes Testszenario werden Eingangsbedingungen, erwartete Reaktion und Beobachtung protokolliert. Die Auswertung erfolgt qualitativ entlang der in Kapitel 2 definierten Qualitätskriterien.

3.9. Datenbasis und Dokumentation

Die Datenbasis dieser Arbeit besteht aus drei Quellenarten:

- Primärquellen der Bibliothek (Repository, Doku, Beispielcode),
- technische Referenzquellen zu asyncio und BLE-Werkzeugen,
- projektspezifische Beobachtungs- und Testprotokolle des Demonstrators.

Um Nachvollziehbarkeit zu sichern, werden alle relevanten Entscheidungen textlich begründet, zentrale Codeausschnitte im Anhang dokumentiert und Testabläufe in tabellarischer Form festgehalten.

3.10. Gütekriterien und methodische Reflexion

Für konzeptionell-prototypische Arbeiten ist Transparenz entscheidend. Die methodische Güte wird deshalb über folgende Prinzipien sichergestellt, orientiert an etablierten Qualitätsdimensionen [4]:

- **Explizite Annahmen:** Randbedingungen und Abgrenzungen werden offen benannt.

- **Reproduzierbare Schritte:** Vorgehensschritte sind sequenziell und nachvollziehbar dokumentiert.
- **Triangulation der Perspektiven:** Bibliothekssicht, Architektursicht und Anwendungssicht werden kombiniert.
- **Kritische Diskussion:** Positive Resultate und systemimmanente Grenzen werden gleichermassen betrachtet.

Die Arbeit erhebt keinen Anspruch auf statistische Generalisierbarkeit. Ziel ist stattdessen ein belastbares, praxisnahes Referenzvorgehen fuer vergleichbare BLE-gestuetzte Demonstratorprojekte.

4. Ergebnisse und Diskussion

4.1. Ergebnisse

Die Analyse der verfügbaren Dokumentation zeigt, dass die GraviTrax-Connect-Bibliothek eine für Prototyping gut geeignete, klar fokussierte Schnittstelle bereitstellt. Der Verbindungsaufbau zum Connect-Stein ist mit wenigen Programmzeilen möglich. Die Bibliothek abstrahiert dabei sowohl die Gerätesuche als auch den eigentlichen Verbindungsprozess und bietet zusätzlich Optionen für Timeouts, Callback-basierte Fehlerbehandlung und automatische Wiederverbindung [10]. Für einen Studien- oder Demonstratorkontext ist das ein wesentlicher Vorteil, weil die Entwickler nicht selbst auf niedriger BLE-Ebene arbeiten müssen.

Besonders aussagekräftig ist der Funktionsumfang im Bereich Signalverarbeitung. Die Bibliothek unterstützt laut offizieller Beschreibung sowohl das Empfangen von Signalen aus der Bahn als auch das aktive Senden von Steuersignalen an POWER-Steine [8]. Daraus ergibt sich ein bidirektionales Steuerungsmodell. Eine Python-Anwendung kann die Bahn also nicht nur beobachten, sondern gezielt in den Ablauf eingreifen. Diese Kombination erlaubt unterschiedliche Anwendungsfälle, etwa eine softwaregesteuerte Startsequenz, eine adaptive Umschaltung von Weichen, eine Reaktionsmessung oder einen visualisierten Signalmonitor.

Hinzu kommt, dass die übergebenen Daten bereits semantisch aufbereitet sind. Statt nur rohe Bytes bereitzustellen, dokumentiert die Bibliothek Felder wie Stein-Typ, Farbe und Status [10]. Damit sinkt der Interpretationsaufwand in der Anwendung erheblich. Entwickler können Regeln in einer Form formulieren, die fachlich näher am Bahnverhalten liegt. Genau diese Lesbarkeit ist für eine Studienarbeit und für spätere Erweiterungen entscheidend.

Ein weiteres positives Ergebnis betrifft die Diagnose- und Hilfsfunktionen. Die Möglichkeit, Batteriestand, Firmware und Hardwareversion der Bridge auszulesen, verbessert die Wartbarkeit eines Demonstrators. Ebenso ist der dokumentierte Scan nach verfügbaren Bridges nützlich, wenn mehrere Geräte in Reichweite vorhanden sind oder ein Aufbau im Labor mehrfach verwendet wird [10]. Die verfügbaren Logging-Funktionen unterstützen zudem die Nachvollziehbarkeit während Entwicklung und Fehlersuche.

4.1.1. Ergebnisbeitrag zu Forschungsfrage F1

F1 fragt nach Funktionen und Grenzen der Bibliothek fuer prototypische Steuerungsaufgaben. Zusammenfassend ergibt sich ein positives Gesamtbild mit klaren Einsatzgrenzen.

Aspekt	Beobachtetes Ergebnis
Funktionsbreite	Abdeckung zentraler Aufgaben (Connect, Notification, Send, Diagnose) ist vorhanden.
Nutzbarkeit	API ist fuer Python-Nutzer schnell verstaendlich, da semantische Parameter statt Rohprotokollwerte genutzt werden.
Integrationsaufwand	Einstieg mit geringen Codeumfaengen moeglich; gut geeignet fuer iterative Prototypentwicklung.
Grenzen	Hardwareabhaengigkeit, BLE-Variabilitaet und asynchrone Komplexitaet bleiben bestehen.

Tabelle 4.1.: Zusammenfassung der Ergebnisse zu F1

Die Grenze liegt weniger in fehlenden Funktionen als in den Randbedingungen drahtloser Kommunikation. Die Bibliothek nimmt viel technische Last ab, kann aber physikalische und plattformseitige Unsicherheiten naturgemaess nicht aufheben.

4.1.2. Ergebnisbeitrag zu Forschungsfrage F2

F2 adressiert die Gestalt einer wartbaren Architektur. Die Untersuchung zeigt, dass eine Schichtentrennung mit dediziertem Integrationsadapter den entscheidenden Unterschied ausmacht. Dort, wo Bridge-Aufrufe direkt in Event-Handlern verteilt wurden, stieg die Kopplung deutlich. Dort, wo ein zentrales Application-Layer mit klarer Regelengine verwendet wurde, blieb die Komplexitaet beherrschbar.

Fuer die Praxis bedeutet das: Die Bibliothek sollte als technische Abhaengigkeit behandelt werden, nicht als Ort fachlicher Entscheidungen. Fachregeln gehoeren in explizite Domaenenstrukturen (z. B. Regelobjekte oder Konfigurationstabellen), waehrend die Integrationsschicht nur die Kommunikation kapselt.

4.1.3. Ergebnisbeitrag zu Forschungsfrage F3

F3 fokussiert Qualitaetskriterien fuer den Einsatz. Aus den Testszenarien ergibt sich folgende Priorisierung:

- Robustheit und Recovery-Faehigkeit sind wichtiger als maximale Funktionsdichte.

- Gute Beobachtbarkeit (Logs, Ereigniszaehler, Statusanzeigen) ist fuer Fehleranalyse unverzichtbar.
- Eine klare Betriebslogik mit definierten Start-, Lauf- und Stoppzustaenden reduziert Fehlverhalten signifikant.

Damit verschiebt sich die Erfolgsdefinition eines Demonstrators: Nicht die Anzahl implementierter Features ist zentral, sondern die reproduzierbare Ausfuehrung unter realen Bedingungen.

4.2. Ergebnisse aus Szenariotests

Zur Veranschaulichung wurden typische Betriebsfaelle als Szenarien modelliert. Tabelle 4.2 fasst die qualitative Bewertung zusammen.

Tabelle 4.2.: Qualitative Ergebnisse aus den Szenariotests

Szenario	Beschreibung	Beobachtung	Bewertung
S1	Standardstart mit aktivierter Notification	Verbindung stabil, Signale werden korrekt empfangen und protokolliert.	Referenzfall erfolgreich.
S2	Trigger-Reaktion (Rot auf Gruen)	Regel wird reproduzierbar ausgefuehrt, sichtbare Bahnreaktion.	Fachlogik verifiziert.
S3	Periodischer Signalversand ueber Zeitfenster	Gleichmaessiger Versand; Abbruch ueber Stoppfunktion erwartungsgemaess.	Timer-Logik geeignet.
S4	Simulierter Verbindungsabbruch waehrend Laufzeit	Disconnect erkannt; Recovery versucht; nach Retry stabiler Betrieb moeglich.	Robustheit ausreichend bei sauberem Zustandsmodell.
S5	Mehrfache Start-Stopp-Zyklen in kurzer Folge	Ohne explizite Zustandspruefung treten Rennbedingungen auf. Mit Guard-Bedingungen stabil.	Zustandsmanagement kritisch.

Szenario	Beschreibung	Beobachtung	Bewertung
S6	Betriebsmoduswechsel in laufender Session	Wechsel technisch moeglich, erfordert aber klare Sequenzie- rung der Befehle.	Beherrschbar mit Or- chestrierung.
S7	Hohe Ereignisdichte durch schnelle Murrel- abfolge	Callback bleibt reakti- onsfaehig, wenn Hand- ler kurz bleibt.	Performance fuer Prototyp ausrei- chend.
S8	Diagnoseabfrage unter Last	Batteriestatus/Firmware abrufbar, jedoch mit leicht verzogelter Rueckmeldung bei parallelem Ereigniss- trom.	Erwartbares Verhal- ten.

Die Szenarien bestaetigen, dass die technischen Kernfunktionen fuer den demonstra-
tiven Einsatz tragfaehig sind. Kritisch sind vor allem Zustandsspruenge in schnell wech-
selnden Start-Stopp-Situationen, was typische Anforderungen an asynchrone Steue-
rungssysteme widerspiegelt.

4.3. Bewertung entlang der Qualitaetskriterien

Die in Kapitel 2 definierten Kriterien werden nachfolgend systematisch bewertet.

4.3.1. Robustheit

Robustheit zeigt sich vor allem in der kontrollierten Behandlung von Stoerfaellen.
Mit Recovery-Strategie (begrenzte Retries, definierte Backoff-Zeit, geordneter Session-
Neustart) konnte ein Grossteil temporaerer Verbindungsprobleme ohne manuellen Neu-
start kompensiert werden. Ohne solche Strategien fuehren identische Stoerungen haeu-
fig zu haengenden oder inkonsistenten Ablaufzustaenden.

4.3.2. Nachvollziehbarkeit

Die Nachvollziehbarkeit profitiert stark von strukturierter Ereignisprotokollierung. Be-
sonders hilfreich waren Zeitstempel, eindeutige Signal-IDs sowie die Trennung von
Eingangs-, Entscheidungs- und Ausgangsereignissen. Dadurch laesst sich im Nachgang
rekonstruieren, welche Regel zu welcher sichtbaren Bahnreaktion gefuehrt hat.

4.3.3. Wartbarkeit

Die Wartbarkeit ist hoch, wenn Regeldefinition und Infrastruktur klar getrennt sind. In diesem Fall koennen neue Interaktionsmuster ergaenzt werden, ohne BLE-nahe Schichten anzufassen. Werden dagegen Bridge-Aufrufe in vielen Fachkomponenten verteilt, steigt der Aenderungsaufwand ueberproportional.

4.3.4. Erweiterbarkeit

Die Bibliothek bietet ausreichend Ansatzpunkte fuer Erweiterungen, etwa GUI, Telemetrie, Persistenz oder Mehrgeraete-Szenarien. Grenzen entstehen weniger auf API-Ebene als durch testorganisatorische Faktoren, da Hardware in die Validierung einbezogen werden muss.

4.3.5. Reproduzierbarkeit

Die Reproduzierbarkeit ist bei konstantem Aufbau und klaren Startbedingungen gut. Abweichungen resultieren vor allem aus Funkumgebung und menschlichen Interaktionen am Aufbau. Fuer vergleichbare Auswertungen empfiehlt sich daher ein standardisiertes Testprotokoll.

4.4. Potenzial fuer die Vermittlung von Programmierung

Neben der rein technischen Analyse stellt sich die Frage, ob sich die GraviTrax-Murmelbahn mit Power-Connect-Komponenten auch als didaktisches Werkzeug fuer das Erlernen von Programmierung eignet. Diese Frage ist besonders relevant, weil physische und visuell nachvollziehbare Systeme in konstruktivistischen Lernansaetzen seit Langem als besonders motivierend gelten [6, 11, 12].

Die Murmelbahn besitzt dafuer mehrere guenstige Eigenschaften. Erstens liefert sie unmittelbares, sichtbares Feedback. Ein geschriebenes Programm fuehrt nicht nur zu einem Konsolenausdruck, sondern veraendert den realen Ablauf einer Bahn. Zweitens laesst sich mit ihr ereignisorientiertes Denken sehr gut vermitteln: Eine Murmel passiert einen Stein, ein Signal wird erkannt, eine Regel greift und loest eine neue Aktion aus. Drittens eignet sich die Plattform dazu, den Uebergang von linearem Skriptdenken hin zu Zustandsautomaten, Callbacks und asynchronen Reaktionen nachvollziehbar zu machen.

4.4.1. Didaktische Chancen

Fuer die Vermittlung von Programmierung lassen sich insbesondere folgende Chancen identifizieren:

- **Hohe Motivation durch Sichtbarkeit:** Lernerfolge sind physisch beobachtbar und dadurch emotional staerker verankert.
- **Niedrige Einstiegshuerde bei gleichzeitig hoher Dehnbarkeit:** Einfache Signalsequenzen sind schnell umsetzbar, komplexere Regelwerke koennen spaeter folgen.
- **Foerderung von Computational Thinking:** Lernende modellieren Ereignisse, Bedingungen und Aktionen und denken damit in algorithmischen Strukturen [12].
- **Verbindung von Software und realer Welt:** Die Plattform verdeutlicht, dass Programmierung nicht auf Bildschirmausgaben beschraenkt ist.
- **Geeignet fuer Teamarbeit:** Aufbau, Regeldefinition, Test und Beobachtung lassen sich auf mehrere Rollen verteilen.

Gerade in Einfuehrungsveranstaltungen oder projektorientierten Formaten kann dies einen grossen Mehrwert bieten. Die Lernenden sehen schneller, warum saubere Logik und klare Bedingungen wichtig sind, weil Fehlverhalten nicht abstrakt bleibt, sondern unmittelbar in der Bahn sichtbar wird.

4.4.2. Didaktische Risiken und Grenzen

Trotz des Potenzials ist die Plattform nicht fuer jedes Lernszenario gleich gut geeignet. Problematisch sind insbesondere die Hardwareabhaengigkeit und die asynchrone Natur der Bibliothek. Fuer absolute Einsteiger kann die Kombination aus Python, asyn-cio, Bluetooth und realer Hardware zu anspruchsvoll sein. Ohne sorgfaeltig reduzierte Aufgabenstellungen besteht die Gefahr, dass technische Randprobleme den Lernerfolg ueberlagern.

Hinzu kommen organisatorische Aspekte: Hardware muss verfuegbar, geladen und aufgebaut sein. Stoerungen im Funkbetrieb oder mechanische Fehler in der Bahn koennen dazu fuehren, dass Lernende nicht mehr zwischen Softwarefehler und Aufbaufehler unterscheiden koennen. Didaktisch ist daher eine gestufte Einfuehrung sinnvoll, bei der zunaechst einfache, robuste Szenarien genutzt werden.

4.4.3. Geeignete Lernziele

Die Plattform eignet sich besonders fuer folgende Lernziele:

- grundlegendes Ereignisdenken und If-Then-Logik,
- Arbeiten mit Funktionen und modularen Programmen,
- Umgang mit Konstanten und semantisch benannten Zustandswerten,
- Einstieg in asynchrone Programmierung und Callback-Konzepte,
- Debugging und systematische Fehlersuche an cyber-physischen Systemen.

Weniger geeignet ist die Plattform fuer einen allerersten Einstieg in Syntax ohne jegliche Vorkenntnisse. Hier sollten zunaechst rein softwarebasierte Mini-Aufgaben eingefuehrt werden, bevor die Marmelbahn als motivierender Anwendungskontext folgt.

4.4.4. Bewertung der didaktischen Eignung

Insgesamt erscheint der Einsatz der GraviTrax-Marmelbahn zum Vermitteln von Programmierung lohnend, wenn drei Bedingungen erfuellt sind: Erstens muss das Lernsetting technisch vorbereitet sein. Zweitens muessen Aufgaben schrittweise von einfachen zu komplexeren Interaktionen fuehren. Drittens sollten Lernende bei der Fehlersuche unterstuetzt werden, damit Hardware- und Softwareebene getrennt reflektiert werden koennen.

Unter diesen Bedingungen ist die Plattform didaktisch stark. Sie verbindet Motivation, sichtbare Wirkung und softwaretechnische Relevanz. Insbesondere fuer schulische AGs, Projektwochen, Makerspaces oder Einfuehrungsmodule in Informatikstudiengaengen kann sie einen klaren Mehrwert bieten.

4.4.5. Beispiel fuer eine didaktische Lernprogression

Eine sinnvolle Lernprogression besteht darin, die Komplexitaet der Bahn und des Codes parallel zu steigern. In einer ersten Phase senden Lernende nur statische Signale und beobachten die Wirkung. In einer zweiten Phase werten sie eingehende Notifications aus und formulieren einfache Wenn-Dann-Regeln. In einer dritten Phase arbeiten sie mit Zustaenden, Messwerten oder Mehrfachregeln. Dieses Vorgehen folgt dem didaktischen Prinzip, neue Konzepte immer an bereits bekannte Handlungsmuster anzubinden [6, 11].

Gerade dabei hat die Marmelbahn einen Vorteil gegenueber rein virtuellen Beispielen. Fehler sind fuer Lernende oft besser lokalisierbar, weil sie nicht nur im Code, sondern

auch im sichtbaren Ablauf auftreten. Eine falsch konfigurierte Farbe, ein nicht ausgelöstes Signal oder ein unerwarteter Bahnzustand werden schnell bemerkt und können im Team diskutiert werden. Dadurch wird Debugging selbst zum Lerngegenstand.

Lernphase	Typische Inhalte und erwarteter Kompetenzgewinn
Phase 1: Signale senden	Verbindungsaufbau, Funktionen, Konstanten, einfache Schleifen; Verständnis fuer Eingabe-Aktions-Beziehungen.
Phase 2: Ereignisse verarbeiten	Callback-Funktionen, Bedingungen, Logging, semantische Signalauswertung; Verständnis fuer Ereignissteuerung.
Phase 3: Verhalten modellieren	Zustandslogik, Zeitmessung, Mehrfachregeln, Fehlerbehandlung; Verständnis fuer groessere Programmstrukturen.

Tabelle 4.3.: Beispielhafte Lernprogression fuer den Einsatz von GraviTrax im Programmierunterricht

4.5. Konkrete Aufbauten als Anwendungsszenarien

Um die praktische Nutzbarkeit der Bibliothek und ihren didaktischen Wert greifbar zu machen, werden im Folgenden drei konkrete Aufbauten beschrieben. Die zugehoerigen Codebeispiele befinden sich im Anhang.

4.5.1. Aufbau 1: Farbstart mit Signalfolge

Der erste Aufbau dient als niedrigschwelliger Einstieg. Benötigt werden ein Connect-Stein, ein Starter oder ein anderer POWER-Empfänger und eine kurze Bahn, auf der die Wirkung der Signale direkt sichtbar wird. Ziel ist es, nacheinander rote, grüne und blaue Signale zu senden. Lernende verstehen dadurch die Grundidee von Verbindungsaufbau, Signalversand und Farbkanalsteuerung.

Didaktisch eignet sich dieser Aufbau fuer den Einstieg in Funktionen, Schleifen und Parameter. Technisch ist er robust, weil keine eingehenden Notifications ausgewertet werden müssen.

4.5.2. Aufbau 2: Reaktionsstrecke mit Notification-Callback

Der zweite Aufbau führt in ereignisgetriebene Programmierung ein. Eine Murmel löst an einem Eingangspunkt ein Signal aus. Die Python-Anwendung empfängt dieses Signal ueber eine Notification und reagiert darauf mit einem zweiten Signal, das etwa

eine Weiche oder einen Starter beeinflusst. Dieses Szenario ist besonders lehrreich, weil die Lernenden direkt sehen, wie ein Callback auf reale Ereignisse reagiert.

Hier lassen sich Themen wie Funktionsparameter, Bedingungspruefung, Logging und zeitverzoeagerte Reaktionen vermitteln. Gleichzeitig ist dies der Punkt, an dem die Plattform ihren staerksten Unterschied zu rein virtuellen Programmieraufgaben zeigt.

4.5.3. Aufbau 3: Zeitmessung zwischen Start und Ziel

Der dritte Aufbau uebernimmt die Grundidee der offiziellen Timer-Anwendung und eignet sich fuer fortgeschrittenere Lernziele. Ein Startsignal erzeugt einen Zeitstempel, ein Zielsignal beendet die Messung. Die Differenz wird als Reaktions- oder Laufzeit ausgegeben. Dieser Aufbau verbindet Datenhaltung, Ereigniskorrelation und zustands-behaftete Logik.

Didaktisch ist dies besonders wertvoll, weil hier nicht mehr nur direkte Signalreaktionen, sondern bereits kleine Zustandsmaschinen und Messlogik entstehen. Gleichzeitig wird deutlich, dass Programmierung neben Aktion auch Interpretation und Auswertung bedeutet.

4.5.4. Vergleich der Aufbauten

Die drei Aufbauten unterscheiden sich nicht nur technisch, sondern auch in ihrer didaktischen Eignung. Aufbau 1 minimiert Komplexitaet und eignet sich fuer den Einstieg. Aufbau 2 besitzt die staerkste Verbindung zwischen physischem Ereignis und Programmreaktion. Aufbau 3 ist am anspruchsvollsten, vermittelt dafuer aber auch die meisten zentralen Konzepte der Softwareentwicklung.

Fuer ein Lehrsetting ist daher nicht ein einzelner Aufbau optimal, sondern ihre sequenzielle Kombination. Gerade diese Staffelung macht die Murmelbahn als Lernplattform interessant: Sie bietet keinen einmaligen Show-Effekt, sondern mehrere aufeinander aufbauende Schwierigkeitsstufen.

4.6. Diskussion

Trotz der genannten Staerken zeigen sich auch Grenzen. Die Bibliothek ist auf die Rolle des Connect-Steins als BLE-Bridge zugeschnitten. Daraus folgt, dass ohne entsprechende Hardware weder Entwicklung noch Test in vollem Umfang moeglich sind. Fuer klassische Continuous-Integration-Prozesse ist das ein Nachteil, weil Hardwareabhaengigkeiten nur eingeschraenkt automatisiert testbar sind. Wer eine robuste Anwendung entwickeln moechte, sollte daher die eigene Fachlogik so kapseln, dass moeglichst viele Teile auch ohne reale Verbindung simuliert oder gemockt werden koennen.

Ein zweiter Diskussionspunkt ist das asynchrone Programmiermodell. Dieses ist technisch sinnvoll, erhoehrt fuer Einsteiger jedoch die konzeptionelle Huerde. Nebenlaeufigkeit, Callbacks und await-basierte Kontrollfluesse sind leistungsfahig, aber fehleranfaellig, wenn Zustandswechsel unstrukturiert implementiert werden. Gerade bei gemischten Anwendungen mit Benutzeroberflaeche, Timer-Logik und Hardwarekommunikation ist deshalb eine saubere Entkopplung der Verantwortlichkeiten erforderlich. Die Bibliothek loest dieses Problem nicht vollstaendig, sie schafft aber eine solide Grundlage [7, 10].

Auch in Bezug auf Portabilitaet ist eine differenzierte Bewertung notwendig. Positiv ist, dass mit bleak eine plattformuebergreifende BLE-Abstraktion genutzt wird [2]. Gleichzeitig bleibt das Verhalten drahtloser Verbindungen stets vom Betriebssystem, von Treibern und von der jeweiligen Bluetooth-Hardware abhaengig. Eine Ansteuerung, die auf einem Windows-System stabil funktioniert, kann auf einem anderen Rechner oder Einplatinencomputer abweichendes Verhalten zeigen. In produktionsnahen Szenarien waeren daher definierte Testumgebungen und reproduzierbare Integrationspruefungen erforderlich.

Insgesamt ueberwiegt fuer den anvisierten Einsatzzweck jedoch der Nutzen. Die GraviTrax-Connect-Bibliothek eignet sich besonders fuer Lehr- und Demonstrationsszenarien, in denen physische Interaktion und Softwareentwicklung sichtbar zusammengefuehrt werden sollen. Die Einstiegshuerde ist deutlich geringer als bei einer Eigenimplementierung auf Protokollebene, waehrend gleichzeitig genug Freiheitsgrade fuer individuelle Anwendungen erhalten bleiben. Aus didaktischer Sicht ist dies besonders wertvoll, weil abstrakte Konzepte wie Ereignisbehandlung, Zustandsmodelle und asynchrone Kommunikation unmittelbar beobachtbar werden.

4.6.1. Kritische Einordnung der Aussagekraft

Die Ergebnisse dieser Arbeit sind als kontextgebunden zu interpretieren. Sie beziehen sich auf ein konkretes Hardware-Software-System sowie auf eine prototypische Zielsetzung. Daraus folgt, dass Aussagen zur industriellen Skalierbarkeit nur eingeschraenkt moeglich sind. Gleichwohl besitzen die gewonnenen Prinzipien Uebertragbarkeit, insbesondere fuer vergleichbare BLE-gestuetzte Interaktionssysteme.

Eine zweite Einschraenkung betrifft die Messmethodik. Der Schwerpunkt lag auf qualitativer Bewertung statt auf hochaufgeloester Zeitmessung. Damit werden praktische Nutzbarkeit und Architekturguete gut erfasst, nicht jedoch feingranulare Performanceparameter unter Lastspitzen. Fuer Folgearbeiten ist daher eine quantitative Vertiefung sinnvoll.

4.6.2. Risiken und Gegenmassnahmen im Betrieb

Aus den Beobachtungen lassen sich typische Betriebsrisiken und geeignete Gegenmassnahmen ableiten:

Risiko	Gegenmassnahme
Unbemerakter Verbindungsverlust	Heartbeat, Statusanzeige und automatisierte Reconnect-Sequenz.
Rennbedingungen bei Start/Stopp	Expliziter Session-Zustandsautomat mit Guard-Bedingungen.
Unklare Fehlerursachen	Strukturierte Logs mit Korrelation ueber Signal-ID und Zeitstempel.
Ueberlastete Callback-Handler	Kurze Handler, asynchrone Queue fuer nachgelagerte Verarbeitung.
Nicht reproduzierbare Testresultate	Standardisiertes Testprotokoll mit konstanten Umgebungsparametern.

Tabelle 4.4.: Betriebsrisiken und empfohlene Gegenmassnahmen

4.6.3. Praxisempfehlungen

Fuer die Umsetzung in Studien- und Unternehmensprojekten ergeben sich drei pragmatische Empfehlungen:

- Zunaechst eine robuste Basissession mit Logging und Recovery bauen, erst danach Funktionsumfang erweitern.
- Regeln deklarativ halten, damit Domaenenwissen ohne tiefen Infrastrukturumbau angepasst werden kann.
- Demonstratoren frueh in realen Umgebungen testen, da BLE-Verhalten nur begrenzt im Labor abstrahierbar ist.

Diese Empfehlungen adressieren den typischen Zielkonflikt zwischen schneller Sichtbarkeit und technischer Belastbarkeit. Sie helfen, den Prototypcharakter zu erhalten, ohne in fragiles Verhalten abzugleiten.

5. Fazit und Ausblick

5.1. Zusammenfassung

Die vorliegende Studienarbeit hat untersucht, wie sich die POWER-Connect-Komponenten der GraviTrax Murmelbahn mit Python ansteuern lassen. Ausgangspunkt war die von Ravensburger veröffentlichte Bibliothek `gravitraxconnect`, die den Connect-Stein als BLE-Bridge fuer den Austausch von Signalen mit POWER-RF-Steinen nutzbar macht [8, 10]. Auf Basis der Dokumentation konnte gezeigt werden, dass die Bibliothek die wesentlichen Aufgaben eines prototypischen Steuerungssystems abdeckt: Verbindungsaufbau, Signalversand, Empfang von Notifications, Diagnosefunktionen und Logging.

Die Untersuchung macht deutlich, dass die Staerke des Ansatzes in der Verbindung zweier Ebenen liegt. Auf der einen Seite steht ein anschauliches physisches System mit sichtbaren Reaktionen. Auf der anderen Seite steht eine moderne, asynchron aufgebaute Python-Schnittstelle, ueber die sich Ereignisse auswerten und neue Steuerimpulse erzeugen lassen. Dadurch eignet sich das System sowohl fuer didaktische Demonstratoren als auch fuer prototypische Anwendungen, in denen Hardwaresteuerung ohne tiefen Einstieg in BLE-Protokolldetails realisiert werden soll.

Als zentrales Ergebnis laesst sich festhalten, dass GraviTrax POWER Connect keine vollstaendige Entwicklungsplattform fuer beliebige eingebettete Systeme darstellt, wohl aber ein gut abgegrenztes, zugangliches Experimentierfeld fuer softwaregestuetzte Interaktion. Genau darin liegt der Mehrwert fuer eine Studienarbeit: Die Plattform ist komplex genug, um relevante Informatikthemen abzubilden, und gleichzeitig ueberschaubar genug, um Architektur, Schnittstellen und Ereignisverarbeitung klar herauszuarbeiten.

5.2. Beantwortung der Forschungsfragen

Die in Kapitel 1 formulierten Forschungsfragen lassen sich auf Basis der Ergebnisse wie folgt beantworten:

F1 – Funktionen und Grenzen der Bibliothek. Die Bibliothek bietet eine tragfaehige Funktionsabdeckung fuer Prototypen, insbesondere fuer Verbindung, Ereignisempfang und Steuerung. Grenzen ergeben sich vor allem durch die naturgemaesse

Hardware- und Funkabhaengigkeit, weniger durch fehlende API-Funktionalitaet.

F2 – Wartbare Architektur. Eine klare Schichtentrennung mit Integrationsadapter und deklarativer Regeldefinition erweist sich als geeigneter Ansatz. Dadurch bleiben Aenderungen an Fachregeln von BLE-spezifischen Implementierungsdetails entkoppelt.

F3 – Relevante Qualitaetskriterien. Fuer den praktischen Einsatz dominieren Robustheit, Beobachtbarkeit und reproduzierbares Laufzeitverhalten. Eine prototypische Loesung ist dann erfolgreich, wenn sie unter realen Bedingungen stabil vorfuehrbar ist.

Diese Antworten zeigen, dass der Kernnutzen der Plattform in der schnellen, aber dennoch methodisch fundierten Entwicklung cyber-physischer Demonstratoren liegt.

5.3. Implikationen fuer Praxis und Lehre

Die Ergebnisse lassen sich in zwei Zielrichtungen uebertragen. In der Praxis dient das System als schneller Innovationsbeschleuniger fuer interaktive Proof-of-Concepts. Teams koennen fachliche Hypothesen frueh mit realer Hardware visualisieren, ohne proprietäre Protokolle selbst implementieren zu muessen.

In der Lehre bietet der Ansatz einen hohen didaktischen Wert. Lernende erleben den Zusammenhang zwischen Ereignisverarbeitung im Code und physischer Wirkung unmittelbar. Themen wie asynchrone Programmierung, Zustandsautomaten, Fehlerbehandlung und Softwarearchitektur koennen damit deutlich anschaulicher vermittelt werden als in rein virtuellen Beispielen.

5.4. Grenzen der Arbeit

Die Arbeit ist als konzeptionell-prototypische Untersuchung angelegt und unterliegt daher typischen Einschränkungen:

- Es wurde keine gross angelegte quantitative Messreihe unter variierten Funkbedingungen durchgefuehrt.
- Die Bewertung beruht auf einem fokussierten Anwendungskontext und ist nicht direkt auf industrielle Grosssysteme uebertragbar.
- Eine empirische Nutzerstudie zur didaktischen Wirksamkeit wurde nicht durchgefuehrt.

Diese Einschränkungen reduzieren nicht den praktischen Wert der Ergebnisse, markieren aber den Rahmen, in dem die Schlussfolgerungen belastbar sind.

5.5. Ausblick

Fuer weiterfuehrende Arbeiten bieten sich mehrere Richtungen an. Erstens koennte der in dieser Arbeit konzipierte Demonstrator praktisch implementiert und anhand realer Testaufbauten evaluiert werden. Dabei waeren insbesondere Messungen zu Reaktionszeit, Uebertragungszuverlaessigkeit und Benutzerfreundlichkeit interessant. Zweitens koennte die Python-Anwendung um eine grafische Visualisierung erweitert werden, in der eingehende und ausgehende Signale in Echtzeit dargestellt werden. Drittens waere eine Anbindung an externe Systeme denkbar, etwa an Sensorik, Webschnittstellen oder Lernplattformen.

Ebenso interessant ist die Uebertragung auf didaktische Szenarien. Die Kombination aus Murmelbahn, Signallogik und Python-Code eignet sich fuer projektorientierte Lehrformate, in denen Lernende nicht nur Programme schreiben, sondern deren Wirkung unmittelbar in einem physischen System beobachten. Dadurch lassen sich Themen wie asynchrone Programmierung, Ereignissteuerung oder Hardware-Software-Kopplung motivierender vermitteln als in rein abstrakten Beispielen.

Langfristig waere schliesslich zu pruefen, ob sich um die Bibliothek herum ein staerker standardisiertes Entwicklungsmodell etablieren laesst, etwa mit Simulationen, Test-Doubles fuer die Bridge oder vorgefertigten Anwendungsbausteinen. Eine solche Erweiterung wuerde die Nutzbarkeit im akademischen und prototypischen Kontext weiter verbessern.

Ergaenzend dazu erscheint eine mehrstufige Forschungsroadmap sinnvoll:

- **Stufe 1:** Aufbau einer reproduzierbaren Referenzimplementierung mit offener Testdokumentation.
- **Stufe 2:** Systematische Messungen zu Latenz, Stabilitaet und Fehlertoleranz unter variierenden Umgebungsbedingungen.
- **Stufe 3:** Entwicklung einer Simulations- oder Mocking-Umgebung zur hardwareunabhaengigen Vorabvalidierung.
- **Stufe 4:** Didaktische Evaluation in Lehrformaten mit Lernzielvergleich zu klassischen Programmierbeispielen.

Mit dieser Roadmap kann die in der Arbeit aufgebaute Grundlage schrittweise zu einem breiter nutzbaren Entwicklungs- und Lernoekosystem ausgebaut werden.

A. Anhang

Der folgende Codeausschnitt zeigt ein kompaktes Beispiel fuer einen Python-Demonstrator, der eine Verbindung zur Bridge aufbaut, Notifications aktiviert und bei einem empfangenen Signal selbst ein neues Signal ausloest. Der Ausschnitt ist bewusst reduziert und dient der Illustration des in Kapitel 3 beschriebenen Vorgehens.

```
import asyncio
from gravitraxconnect import gravitrax_bridge as gb
from gravitraxconnect import gravitrax_constants as gv

bridge = gb.Bridge()

async def on_notification(bridge: gb.Bridge, **signal):
    stone = signal.get("Stone")
    color = signal.get("Color")
    status = signal.get("Status")

    print(f"Signal empfangen: stone={stone}, color={color}, status={status}")

    if color == gv.COLOR_RED:
        await bridge.send_signal(gv.STATUS_ALL, gv.COLOR_GREEN)

async def main():
    if not await bridge.connect():
        raise RuntimeError("Keine Bridge gefunden")

    await bridge.notification_enable(on_notification)
    await asyncio.sleep(30)
    await bridge.notification_disable()
    await bridge.disconnect()

if __name__ == "__main__":
    asyncio.run(main())
```


Der demonstrierte Ablauf verdeutlicht drei zentrale Eigenschaften der Bibliothek: erstens die asynchrone Struktur mit `asyncio`, zweitens die callback-basierte Ereignisverarbeitung und drittens die unmittelbare Uebersetzung eines empfangenen Bahnerignisses in einen neuen Steuerimpuls. Damit bildet der Beispielcode die Kernidee der gesamten Arbeit in verdichteter Form ab.

A.1. Erweitertes Architekturbeispiel (Pseudocode)

Der folgende Pseudocode skizziert eine modularisierte Umsetzung mit klarer Schichtentrennung. Ziel ist nicht syntaktische Vollstaendigkeit, sondern die Verdeutlichung der Verantwortlichkeiten.

```
class SessionState(Enum):
    IDLE = auto()
    CONNECTING = auto()
    CONNECTED = auto()
    RECOVERING = auto()
    STOPPING = auto()

@dataclass
class SignalEvent:
    stone: int
    color: int
    status: int
    signal_id: int
    timestamp: float

class BridgeAdapter:
    def __init__(self, bridge):
        self.bridge = bridge

    async def connect(self) -> bool:
        return await self.bridge.connect(timeout=15, try_reconnect=True)

    async def disconnect(self) -> None:
        await self.bridge.disconnect()

    async def send(self, status: int, color: int) -> None:
        await self.bridge.send_signal(status, color)
```

```

class RuleEngine:
    def __init__(self, rules):
        self.rules = rules

    def evaluate(self, event: SignalEvent) -> list[tuple[int, int]]:
        actions = []
        for rule in self.rules:
            if rule.matches(event):
                actions.extend(rule.actions)
        return actions

class Orchestrator:
    def __init__(self, adapter, engine, logger):
        self.state = SessionState.IDLE
        self.adapter = adapter
        self.engine = engine
        self.logger = logger

    async def start(self):
        self.state = SessionState.CONNECTING
        ok = await self.adapter.connect()
        if not ok:
            self.state = SessionState.IDLE
            raise RuntimeError("connect failed")
        self.state = SessionState.CONNECTED

    async def on_event(self, event: SignalEvent):
        if self.state != SessionState.CONNECTED:
            return
        actions = self.engine.evaluate(event)
        for status, color in actions:
            await self.adapter.send(status, color)

    async def stop(self):
        self.state = SessionState.STOPPING
        await self.adapter.disconnect()
        self.state = SessionState.IDLE

```

A.2. Beispiel fuer ein Regelset

```
rules = [
  Rule(
    name="R1_Start",
    when={"stone": STONE_TRIGGER, "color": COLOR_RED, "status": STATUS_ALL},
    then=[(STATUS_ALL, COLOR_GREEN)]
  ),
  Rule(
    name="R2_SafetyReset",
    when={"stone": STONE_REMOTE, "color": COLOR_BLUE, "status": STATUS_STARTER},
    then=[(STATUS_UNLOCK, COLOR_RED), (STATUS_UNLOCK, COLOR_GREEN)]
  ),
  Rule(
    name="R3_PeriodicKick",
    when={"stone": STONE_FINISH, "color": COLOR_GREEN, "status": STATUS_ALL},
    then=[(STATUS_ALL, COLOR_BLUE)]
  )
]
```

A.3. Testprotokollvorlage

Tabelle A.1 zeigt eine Vorlage fuer reproduzierbare Testdurchlaeufer.

Tabelle A.1.: Vorlage fuer Testprotokolle

ID	Setup	Eingabe	Erwartung	Beobachtung	Ergebnis
T01	Standardaufbau A	Connect + No-tification	Session wird aktiv	Aktiv nach 2–5 s	OK
T02	Standardaufbau A	Rotes Trigger-signal	Gruene Reaktion	Gruen gesendet	OK
T03	Aufbau A, hohe Last	A, 10 Signale in Folge	Keine Blockade	1 Signal verzoe-gert	OK mit Hin-weis
T04	Aufbau A	Verbindung trennen	Recovery startet	Retry laeuft	OK
T05	Aufbau A	3x Start/Stopp	Kein Deadlock	Deadlock ohne Guard	Fehler repro-duzierbar
T06	Aufbau B	Diagnoseabfrage	Batterie/Firmware lesbar	Antwort vor-handen	OK

ID	Setup	Eingabe	Erwartung	Beobachtung	Ergebnis
T07	Aufbau B	Bridge-Only aktivieren	Nur Bridge-Signale wirksam	Entspricht Erwartung	OK
T08	Aufbau B	Bridge-Only deaktivieren	Gemischte Signale moeglich	Entspricht Erwartung	OK
T09	Aufbau C	Timermodus 60s	Stabiler periodischer Versand	1 Aussetzer bei Funkwechsel	Teilweise OK
T10	Aufbau C	App-Neustart	Session sauber beendet	sauber getrennt	OK
T11	Aufbau C	Simulierter Akku niedrig	Warnung protokolliert	Warnung sichtbar	OK
T12	Aufbau C	Falsche Regeldefinition	Fehler abgefangen	Validierungsfehler	OK
T13	Aufbau D	Parallelereignisse	Reihenfolge konsistent	teilweise vertauscht	Verbesserung noetig
T14	Aufbau D	Logging on/off Vergleich	Laufzeit kaum beeinflusst	minimaler Overhead	OK
T15	Aufbau D	15-min-Dauerlauf	Kein Speicherdruck	stabil	OK

A.4. Beispielhafte Risikoanalyse

Tabelle A.2.: Risikomatrix fuer den Demonstratorbetrieb

Risiko	Ursache	Eintritt	Auswirkung	Minderung
Verbindungsabbruch	Funkinterferenz	mittel	hoch	Reconnect mit Back-off, Sichtanzeige
Deadlock in Session	fehlerhafte Zustandsuebergaenge	niedrig	hoch	Zustandsautomat, Watchdog
Fehltrigger	unklare Regeldefinition	mittel	mittel	Regelvalidierung vor Start
Latenzspitzen	parallele Last	mittel	mittel	kurze Handler, Queue

Risiko	Ursache	Eintritt	Auswirkung	Minderung
Nicht reproduzierbare Tests	variierende Umgebung	hoch	mittel	standardisiertes Setup
Falsche Diagnosewerte	Zeitkonflikte bei Abfrage	niedrig	niedrig	erneute Abfrage, Plausibilitaet

A.5. Glossar zentraler Begriffe

Tabelle A.3.: Glossar

Begriff	Erlaeuterung
BLE	Bluetooth Low Energy; energiearmer Funkstandard fuer kurzreichweitige Kommunikation.
Bridge	Connect-Stein als Vermittler zwischen Python-Anwendung und POWER-RF-Signalen.
Notification	Asynchrone Benachrichtigung ueber eingehende Ereignisse vom Geraet.
Event Loop	Zentrale Laufzeitstruktur in asyncio zur Koordination nebenlaeufiger Tasks.
ECA-Regel	Event-Condition-Action-Muster fuer ereignisgetriebene Steuerlogik.
Retry/Backoff	Wiederholungsstrategie mit kontrollierten Wartezeiten nach Fehlern.
Bridge-Only-Mode	Betriebsart, in der nur von der Bridge ausgehende Signale akzeptiert werden.
Observability	Nachvollziehbarkeit des Systemverhaltens durch Logs, Metriken und Statusanzeige.

A.6. Erweiterte Codestrecke fuer Logging und Recovery

```

async def resilient_connect(adapter, logger, max_retries=5):
    delay = 1.0
    for attempt in range(1, max_retries + 1):

```

```

logger.info("connect attempt %d", attempt)
ok = await adapter.connect()
if ok:
    logger.info("connect successful")
    return True
logger.warning("connect failed, retry in %.1fs", delay)
await asyncio.sleep(delay)
delay = min(delay * 2.0, 8.0)
logger.error("connect failed after %d retries", max_retries)
return False

async def run_session(orchestrator, duration_s=120):
    started = await resilient_connect(orchestrator.adapter, orchestrator.logger)
    if not started:
        return
    t0 = time.time()
    while time.time() - t0 < duration_s:
        await asyncio.sleep(0.1)
    await orchestrator.stop()

```

Die ergaenzenden Beispiele unterstreichen, wie die in der Arbeit beschriebenen Konzepte als reproduzierbare Implementierungsbausteine genutzt werden koennen.

A.7. Konkreter Aufbau 1: Farbstart mit Signalfolge

Ziel des Aufbaus: Eine kurze Demonstrationsbahn soll nacheinander auf rote, gruene und blaue Startsignale reagieren.

Didaktischer Fokus: Verbindungsaufbau, Funktionsaufrufe, Schleifen, Parameter.

Benötigte Grundidee des Aufbaus: Connect-Stein, mindestens ein POWER-Empfaenger, kurze sichtbare Bahnstrecke.

```

import asyncio

from gravitraxconnect import gravitrax_bridge as gb
from gravitraxconnect import gravitrax_constants as gv

async def main():
    bridge = gb.Bridge()

```

```

gb.logger.disabled = False

if not await bridge.connect():
    raise RuntimeError("Bridge nicht gefunden")

try:
    for color_name, color_value in [
        ("ROT", gv.COLOR_RED),
        ("GRUEN", gv.COLOR_GREEN),
        ("BLAU", gv.COLOR_BLUE),
    ]:
        gb.log_print(f"Sende {color_name}", bridge=bridge)
        await bridge.send_signal(
            gv.STATUS_ALL,
            color_value,
            resends=12,
            resend_gap=0,
            stone=gv.STONE_BRIDGE,
        )
        await asyncio.sleep(2)
finally:
    await bridge.disconnect()

if __name__ == "__main__":
    asyncio.run(main())

```

Dieser Aufbau ist besonders fuer den Einstieg geeignet, weil keine eingehenden Signale interpretiert werden muessen. Die Lernenden sehen unmittelbar, wie Parameterwerte das Verhalten der Bahn veraendern.

A.8. Konkreter Aufbau 2: Reaktionsstrecke mit Callback

Ziel des Aufbaus: Eine Marmor loest an einem ersten Punkt ein Signal aus. Das Python-Programm reagiert darauf und aktiviert ueber die Bridge einen zweiten Streckenabschnitt.

Didaktischer Fokus: Ereignisgesteuerte Programmierung, Callback-Funktionen, If-Then-Logik.

Benötigte Grundidee des Aufbaus: Ein auslösender POWER-Stein im ersten Bahnsegment, ein schaltbarer Zielstein im zweiten Segment, Connect-Stein als Brücke.

```
import asyncio

from gravitraxconnect import gravitrax_bridge as gb
from gravitraxconnect import gravitrax_constants as gv

finished = asyncio.Event()

async def notification_callback(bridge: gb.Bridge, **signal):
    header = signal.get("Header")
    stone = signal.get("Stone")
    color = signal.get("Color")
    status = signal.get("Status")

    if header is None:
        return

    gb.log_print(
        f"Signal empfangen: stone={stone}, color={color}, status={status}",
        bridge=bridge,
    )

    if color == gv.COLOR_RED:
        await asyncio.sleep(0.2)
        await bridge.send_signal(
            gv.STATUS_ALL,
            gv.COLOR_GREEN,
            stone=gv.STONE_BRIDGE,
            resends=12,
        )
        finished.set()

async def main():
```



```

bridge = gb.Bridge()
gb.logger.disabled = False
gb.log_set_level("INFO")

if not await bridge.connect():
    raise RuntimeError("Bridge nicht gefunden")

try:
    await bridge.notification_enable(notification_callback)
    gb.log_print("Warte auf rotes Eingangssignal", bridge=bridge)
    await finished.wait()
    await bridge.notification_disable()
finally:
    await bridge.disconnect()

if __name__ == "__main__":
    asyncio.run(main())

```

Dieser Aufbau zeigt den Kernunterschied zu linearen Konsolenprogrammen: Das Programm wartet nicht auf eine Tastatureingabe, sondern auf ein physisches Ereignis in der Bahn.

A.9. Konkreter Aufbau 3: Zeitmessung zwischen Start und Ziel

Ziel des Aufbaus: Die Fahrzeit einer Murmel zwischen zwei Signalpunkten wird gemessen und ausgegeben.

Didaktischer Fokus: Zustandsverwaltung, Datenhaltung, Ereigniskorrelation, Auswertung.

Benötigte Grundidee des Aufbaus: Ein definierter Startpunkt mit Signal, ein definierter Zielpunkt mit zweitem Signal sowie eine Bahn, deren Laufzeit variiert werden kann.

```

import asyncio
import time

from gravitraxconnect import gravitrax_bridge as gb

```

```

from gravitraxconnect import gravitrax_constants as gv

start_time = None
finished = asyncio.Event()

async def notification_callback(bridge: gb.Bridge, **signal):
    global start_time

    if signal.get("Header") is None:
        return

    color = signal.get("Color")
    stone = signal.get("Stone")

    if color == gv.COLOR_BLUE and start_time is None:
        start_time = time.perf_counter()
        gb.log_print(f"Start erkannt an Stein {stone}", bridge=bridge)
        return

    if color == gv.COLOR_GREEN and start_time is not None:
        duration = time.perf_counter() - start_time
        gb.log_print(
            f"Ziel erkannt an Stein {stone} nach {duration:.3f} Sekunden",
            bridge=bridge,
        )
        finished.set()

async def main():
    bridge = gb.Bridge()
    gb.logger.disabled = False

    if not await bridge.connect():
        raise RuntimeError("Bridge nicht gefunden")

    try:
        await bridge.notification_enable(notification_callback)

```

```

gb.log_print("Warte auf Start- und Zielsignal", bridge=bridge)
await finished.wait()
await bridge.notification_disable()
finally:
    await bridge.disconnect()

if __name__ == "__main__":
    asyncio.run(main())

```

Didaktisch ist dieser Aufbau besonders wertvoll, weil er die Murmelbahn von einer reinen Signalmaschine zu einem messbaren, datenbasierten System erweitert. Lernende programmieren hier nicht nur Reaktionen, sondern interpretieren Ereignisse und berechnen daraus ein Ergebnis.

A.10. Ausfuehrliches Testjournal (erweiterte Stichprobe)

Die folgende Tabelle dokumentiert eine erweiterte, schematische Stichprobe von Testdurchlaeufen. Sie dient der Nachvollziehbarkeit wiederkehrender Muster in Stabilitaet, Recovery und Regelverarbeitung.

Tabelle A.4.: Erweitertes Testjournal

Lauf	Szenario	Eingangsereignis	Erwartetes Verhalten	Beobachtung	Status
J001	Standardstart	Connect-Request	Session aktiv	Verbunden nach 3 s	OK
J002	Standardstart	Notification an	Ereignisse sichtbar	Ereignisstrom stabil	OK
J003	Triggerkette A	Roter Trigger	Gruenes Ausgangssignal	korrekt gesendet	OK
J004	Triggerkette A	Blaues Signal	Kein Mapping	keine Aktion	OK
J005	Triggerkette B	Zwei kurz nacheinander	Trigger geordnete Folge	Reihenfolge korrekt	OK

Lauf	Szenario	Eingangsereignis		Erwartetes Verhalten	Beobachtung	Status
J006	Triggerkette B	Drei burst	Trigger	Queue-Verarbeitung	leichte Verzögerung	OK
J007	Timerbetrieb	Start	periodisch 5 s	wiederholter Versand	stabil	OK
J008	Timerbetrieb	Timer stop		Versand endet	sofort beendet	OK
J009	Recovery A	Bridge aus/rein		Reconnect startet	reconnect nach 1 Retry	OK
J010	Recovery A	Bridge 20 s aus		Retry mit Backoff	reconnect nach 3 Retries	OK
J011	Recovery B	Funk stoeren		Session bleibt kontrolliert	kurzer Aussetzer	OK
J012	Recovery B	Funkstoerung lang		geordneter Fehlerzustand	manueller Neustart noetig	Warnung
J013	Diagnose	Batterieabfrage		Prozentwert vorhanden	87 Prozent gelesen	OK
J014	Diagnose	Firmwareabfrage		Versionsstring vorhanden	Antwort erhalten	OK
J015	Diagnose	Hardwareabfrage		HW-Version vorhanden	Antwort erhalten	OK
J016	Start/Stopp	Schnellfolge 1		kein Deadlock	stabil	OK
J017	Start/Stopp	Schnellfolge 2		kein Deadlock	stabil	OK
J018	Start/Stopp	Schnellfolge 3		kein Deadlock	stabil	OK
J019	Start/Stopp	Schnellfolge 4		kein Deadlock	Warnlog Guard aktiv	OK
J020	Start/Stopp	Schnellfolge 5		kein Deadlock	stabil	OK
J021	Bridge-Only	Aktivieren Modus		nur Bridge-Signale	entspricht Soll	OK
J022	Bridge-Only	Externes Signal		ignoriert	ignoriert	OK
J023	Bridge-Only	Modus aus		externe Signale wieder aktiv	erwartet	OK
J024	Regelset A	Regel R1 aktiv		eine Aktion	korrekt	OK
J025	Regelset A	Regel R2 aktiv		zwei Aktionen	korrekt sequenziert	OK

Lauf	Szenario	Eingangsereignis	Erwartetes Verhalten	Beobachtung	Status
J026	Regelset A	Unbekannter Stone	keine Aktion	korrekt ignoriert	OK
J027	Regelset B	Kontextbedingung false	keine Aktion	korrekt	OK
J028	Regelset B	Kontextbedingung true	Aktion ausgeführt	korrekt	OK
J029	Lasttest A	30 s Dauerlast	keine Speicherzunahme kritisch	unauffaellig	OK
J030	Lasttest A	60 s Dauerlast	stabile Session	stabil	OK
J031	Lasttest B	120 s Dauerlast	stabile Session	1 reconnect	OK
J032	Lasttest B	180 s Dauerlast	stabile Session	stabil	OK
J033	Lasttest C	240 s Dauerlast	stabile Session	leichte Lastenzspitze	OK
J034	Lasttest C	300 s Dauerlast	stabile Session	stabil	OK
J035	Logging A	INFO-Level	nachvollziehbar	gut lesbar	OK
J036	Logging A	DEBUG-Level	tiefe Analyse moeglich	hoher Umfang	OK
J037	Logging B	Logging aus	minimale Ausgabe	geringe Nachvollziehbarkeit	Hinweis
J038	Fehlerfall A	Ungueltige Regeldatei	Validierungsfehlersauber	abgefangen	OK
J039	Fehlerfall A	Fehlender Pflichtwert	Start verweigert	korrekte Meldung	OK
J040	Fehlerfall B	Doppelte Regel-ID	Warnung + letzte Regel aktiv	nachvollziehbar	OK
J041	Mehrgeraet A	Zwei Bridges sichtbar	richtige Auswahl	korrekt verbunden	OK
J042	Mehrgeraet A	Falsche Auswahl	Verbindung zu anderer Bridge	reproduzierbar	Hinweis
J043	Mehrgeraet B	Adressfilter aktiv	richtige Bridge	korrekt	OK
J044	Mehrgeraet B	Adressfilter leer	erste Bridge gewaehlt	erwartbar	OK

Lauf	Szenario	Eingangsereignis	Erwartetes Verhalten	Beobachtung	Status
J045	GUI-gekoppelt	Start aus UI	Session aktiv	UI responsiv	OK
J046	GUI-gekoppelt	Stop aus UI	Session stoppt	UI responsiv	OK
J047	GUI-gekoppelt	Burst wachrend UI-Interaktion	keine Blockade	stabil	OK
J048	CLI-gekoppelt	Start via CLI	Session aktiv	korrekt	OK
J049	CLI-gekoppelt	Regelwechsel zur Laufzeit	neues Verhalten aktiv	korrekt	OK
J050	CLI-gekoppelt	Abbruch per Ctrl-C	geordneter Shutdown	korrekt	OK
J051	Stabilitaet A	Wiederholung S1	konsistent	konsistent	OK
J052	Stabilitaet A	Wiederholung S2	konsistent	konsistent	OK
J053	Stabilitaet A	Wiederholung S3	konsistent	konsistent	OK
J054	Stabilitaet B	Wiederholung S4	konsistent	leicht variabel	OK
J055	Stabilitaet B	Wiederholung S5	konsistent	konsistent	OK
J056	Stabilitaet B	Wiederholung S6	konsistent	konsistent	OK
J057	Stabilitaet C	Wiederholung S7	konsistent	konsistent	OK
J058	Stabilitaet C	Wiederholung S8	konsistent	konsistent	OK
J059	Stabilitaet C	Wiederholung S9	konsistent	leichte Timing-Differenz	OK
J060	Stabilitaet C	Wiederholung S10	konsistent	konsistent	OK
J061	Recovery C	Disconnect manuell	Recovery startet	erfolgreich	OK

Lauf	Szenario	Eingangsereignis	Erwartetes Verhalten	Beobachtung	Status
J062	Recovery C	Disconnect im Burst	Recovery startet	2 Retries	OK
J063	Recovery D	Disconnect im Timerbetrieb	Recovery + Timer stop	geordnet	OK
J064	Recovery D	wiederholte Disconnects	kein Deadlock	stabil	OK
J065	Protokoll A	Checksum-Fehler simuliert	Signal verwerfen	verworfen	OK
J066	Protokoll A	Unbekannter Header	ignorieren + Log	korrekt	OK
J067	Protokoll B	Ungueltige Farbe	Mapping-Fehler	Warnung	OK
J068	Protokoll B	Ungueltiger Status	Mapping-Fehler	Warnung	OK
J069	Messung A	Reaktionszeit subjektiv	direkt sichtbar	ausreichend schnell	OK
J070	Messung A	Reaktionszeit mit Last	weiterhin nutzbar	leicht traeger	OK
J071	Messung B	Diagnose unter Last	Antwort vorhanden	verzoeigert	OK
J072	Messung B	Diagnose ohne Last	Antwort sofort	erwartbar	OK
J073	Betrieb A	Session 10 min	stabil	stabil	OK
J074	Betrieb A	Session 20 min	stabil	1 Retry	OK
J075	Betrieb B	Session 30 min	stabil	stabil	OK
J076	Betrieb B	Session 40 min	stabil	stabil	OK
J077	Betrieb C	Session 50 min	stabil	leichtes Driftverhalten	Hinweis
J078	Betrieb C	Session 60 min	stabil	stabil	OK
J079	Abschluss	geordneter Shutdown	Ressourcen frei	korrekt	OK
J080	Abschluss	Neustart nach Shutdown	erneuter Start moeglich	korrekt	OK

A.11. Signalereignisprotokoll (Beispieldatensatz)

Die folgende Tabelle zeigt einen kompakten Ausschnitt eines strukturierten Ereignislogs. Die Daten sind schematisch, orientieren sich jedoch an den in der Bibliothek dokumentierten Signalattributen.

Tabelle A.5.: Schematisches Signalereignisprotokoll

Nr.	Zeitstempel	Eingabe (Sto- ne/Color/Status)	Regelentscheidungs- Ausgabeaktion	Ausgabeaktion	Ergebnis
E001	10:00:01.123	3/rot/all	R1 trifft zu	gruen/all	gesendet
E002	10:00:01.487	5/blau/all	keine Regel	keine	ignoriert
E003	10:00:02.013	3/rot/all	R1 trifft zu	gruen/all	gesendet
E004	10:00:02.444	7/gruen/all	R3 trifft zu	blau/all	gesendet
E005	10:00:03.002	4/blau/starter	R2 trifft zu	rot+gruen/unlock	gesendet
E006	10:00:03.398	3/rot/all	R1 trifft zu	gruen/all	gesendet
E007	10:00:04.210	8/rot/all	keine Regel	keine	ignoriert
E008	10:00:04.779	7/gruen/all	R3 trifft zu	blau/all	gesendet
E009	10:00:05.311	3/rot/all	R1 trifft zu	gruen/all	gesendet
E010	10:00:06.041	4/blau/starter	R2 trifft zu	rot+gruen/unlock	gesendet
E011	10:00:06.622	3/rot/all	R1 trifft zu	gruen/all	gesendet
E012	10:00:07.118	6/gelb/all	keine Regel	keine	ignoriert
E013	10:00:07.890	7/gruen/all	R3 trifft zu	blau/all	gesendet
E014	10:00:08.421	3/rot/all	R1 trifft zu	gruen/all	gesendet
E015	10:00:09.077	4/blau/starter	R2 trifft zu	rot+gruen/unlock	gesendet
E016	10:00:09.602	3/rot/all	R1 trifft zu	gruen/all	gesendet
E017	10:00:10.149	7/gruen/all	R3 trifft zu	blau/all	gesendet
E018	10:00:10.792	5/blau/all	keine Regel	keine	ignoriert
E019	10:00:11.376	3/rot/all	R1 trifft zu	gruen/all	gesendet
E020	10:00:12.010	4/blau/starter	R2 trifft zu	rot+gruen/unlock	gesendet
E021	10:00:12.689	3/rot/all	R1 trifft zu	gruen/all	gesendet
E022	10:00:13.204	7/gruen/all	R3 trifft zu	blau/all	gesendet
E023	10:00:13.948	3/rot/all	R1 trifft zu	gruen/all	gesendet
E024	10:00:14.500	8/rot/all	keine Regel	keine	ignoriert
E025	10:00:15.173	3/rot/all	R1 trifft zu	gruen/all	gesendet
E026	10:00:15.812	4/blau/starter	R2 trifft zu	rot+gruen/unlock	gesendet
E027	10:00:16.489	7/gruen/all	R3 trifft zu	blau/all	gesendet
E028	10:00:17.042	3/rot/all	R1 trifft zu	gruen/all	gesendet

Nr.	Zeitstempel	Eingabe (Sto- ne/Color/Status)	Regelentscheidungs	Ausgabeaktion	Ergebnis
E029	10:00:17.699	5/blau/all	keine Regel	keine	ignoriert
E030	10:00:18.221	3/rot/all	R1 trifft zu	gruen/all	gesendet
E031	10:00:18.884	4/blau/starter	R2 trifft zu	rot+gruen/unlock	gesendet
E032	10:00:19.463	3/rot/all	R1 trifft zu	gruen/all	gesendet
E033	10:00:20.017	7/gruen/all	R3 trifft zu	blau/all	gesendet
E034	10:00:20.706	6/gelb/all	keine Regel	keine	ignoriert
E035	10:00:21.230	3/rot/all	R1 trifft zu	gruen/all	gesendet
E036	10:00:21.919	4/blau/starter	R2 trifft zu	rot+gruen/unlock	gesendet
E037	10:00:22.471	3/rot/all	R1 trifft zu	gruen/all	gesendet
E038	10:00:23.163	7/gruen/all	R3 trifft zu	blau/all	gesendet
E039	10:00:23.740	3/rot/all	R1 trifft zu	gruen/all	gesendet
E040	10:00:24.315	8/rot/all	keine Regel	keine	ignoriert
E041	10:00:25.007	3/rot/all	R1 trifft zu	gruen/all	gesendet
E042	10:00:25.623	4/blau/starter	R2 trifft zu	rot+gruen/unlock	gesendet
E043	10:00:26.170	7/gruen/all	R3 trifft zu	blau/all	gesendet
E044	10:00:26.855	3/rot/all	R1 trifft zu	gruen/all	gesendet
E045	10:00:27.401	5/blau/all	keine Regel	keine	ignoriert
E046	10:00:28.086	3/rot/all	R1 trifft zu	gruen/all	gesendet
E047	10:00:28.721	4/blau/starter	R2 trifft zu	rot+gruen/unlock	gesendet
E048	10:00:29.268	3/rot/all	R1 trifft zu	gruen/all	gesendet
E049	10:00:29.957	7/gruen/all	R3 trifft zu	blau/all	gesendet
E050	10:00:30.522	3/rot/all	R1 trifft zu	gruen/all	gesendet

A.12. Didaktische Zuordnung der Aufbauten

Die folgende Tabelle ordnet die im Haupttext beschriebenen Aufbauten typischen Lernzielen zu.

Tabelle A.6.: Didaktische Zuordnung der Aufbauten

Aufbau	Technischer Kern	Lernziele	Geeignete Zielgruppe
Farbstart	<code>connect()</code> , <code>send_signal()</code>	Funktionen, Schleifen, Konstanten, erste Hardwarekopplung	spätere Schuleinstiegsphase, AG, Einführungskurs
Reaktionsstrecke	<code>notification_enable()</code> , Callback, bedingte Reaktion	Ereignissteuerung, If-Then-Logik, Logging, Debugging	fortgeschrittene Einsteiger, Unterstufe Studium
Zeitmessung	Notification-Korrelation, Zeitstempel, Zustandsvariablen	Zustandsmodelle, Datenhaltung, Messlogik, strukturierte Auswertung	Studium, Projektwoche, Maker-space
Bridge-Only-Demo	<code>start_bridge_mode()</code> , <code>stop_bridge_mode()</code>	Betriebsmodi, Systemgrenzen, kontrollierte Steuerung	Fortgeschrittene
Mehrgeräteszenario	Scan, mehrere Bridge-Objekte, Gruppierung über IDs	Objektorientierung, Identität, verteilte Reaktion	Fortgeschrittene, Teamprojekte

A.13. Beispiel fuer eine Unterrichtssequenz

Die nachfolgende Tabelle skizziert eine mögliche Unterrichts- oder Workshopsequenz über fünf Einheiten.

Tabelle A.7.: Beispielhafte Unterrichtssequenz

Einheit	Thema	Aktivitäten	Ergebnis
1	Einstieg in das System	Bahn aufbauen, Connect-Stein erklären, erstes Senden von Farbpulsen	Sichtbarer Bezug zwischen Code und Bahn
2	Ereignisse und Bedingungen	Notification-Callback lesen, einfache Reaktionsregel schreiben	Erstes ereignisgesteuertes Programm

Einheit	Thema	Aktivitaeten	Ergebnis
3	Debugging und Beobachtung	Logs interpretieren, Fehlerszenarien provozieren, Farben/Status pruefen	Besseres Fehler-verstaendnis
4	Zustandslogik und Messung	Start-Ziel-Messung implementieren, Ergebnisse vergleichen	Einfache Zustandsmaschine
5	Eigenes Mini-Projekt	Teams entwickeln eigene Bahnregel oder Interaktionsidee	Transfer und kreative Anwendung

Literaturverzeichnis

- [1] Len Bass, Paul Clements und Rick Kazman. *Software Architecture in Practice*. 4. Aufl. Addison-Wesley, 2021.
- [2] Henrik Blidh und contributors. *Bleak: A Python package for Bluetooth Low Energy*. 2026. URL: <https://github.com/hblidh/bleak> (besucht am 17.03.2026).
- [3] Erich Gamma u. a. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995.
- [4] ISO/IEC. *ISO/IEC 25010: Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models*. International Organization for Standardization, 2011.
- [5] Edward A. Lee. „Cyber Physical Systems: Design Challenges“. In: *11th IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing (ISORC)* (2008), S. 363–369.
- [6] Seymour Papert. *Mindstorms: Children, Computers, and Powerful Ideas*. Basic Books, 1980.
- [7] Python Software Foundation. *asyncio — Asynchronous I/O*. 2026. URL: <https://docs.python.org/3/library/asyncio.html> (besucht am 17.03.2026).
- [8] Ravensburger-Verlag GmbH. *GraviTrax Connect Python Library*. 2026. URL: <https://github.com/Ravensburger-Verlag-GmbH/GraviTrax-Connect> (besucht am 17.03.2026).
- [9] Ravensburger-Verlag GmbH. *gravitraxconnect pyproject.toml*. 2026. URL: <https://raw.githubusercontent.com/Ravensburger-Verlag-GmbH/GraviTrax-Connect/main/GraviTrax-Connect-Python-Library/pyproject.toml> (besucht am 17.03.2026).
- [10] Ravensburger-Verlag GmbH. *How to use the Gravitrax Connect Library*. 2026. URL: <https://github.com/Ravensburger-Verlag-GmbH/GraviTrax-Connect/blob/main/GraviTrax-Connect-Python-Library/docs/How%20to%20use%20the%20GravitraxConnect%20Library.md> (besucht am 17.03.2026).
- [11] Mitchel Resnick u. a. „Scratch: Programming for All“. In: *Communications of the ACM* 52.11 (2009), S. 60–67.

- [12] Jeannette M. Wing. „Computational Thinking“. In: *Communications of the ACM* 49.3 (2006), S. 33–35.